大模型项目面试准备手册

核心原理 实现细节 深入追问

陶壮

2026-11-02

目录

00 阅读方法与项目地图 5
1 先建立完整项目地图 5
2 阅读时识别四种证据状态 5
3 分三轮学 6
4 每个项目都应能回答的八件事 6
5 技术栈应该与具体问题绑定 6
6 如何使用本地证据 6
01 推理系统：OmniInfer、OmniPlacement 与 Omni-EPLB 7
1. 先把三个名字、项目目标和个人边界讲明白 7
2. 理解实现前必须掌握的概念 7
3. 沿一条请求和一个 MoE 层走完执行链 8
4. 数学对象、贪心结论与复杂度边界 9
5. 动态路径：从物理计数到候选和安全切换 10
6. Python、C++、线程与设备内存怎样共同工作 11
7. 本人具体做过什么，证据如何落到代码 12
8. 十二组分级追问：回答后还能再往下讲两层 13
9. 两个纸笔或小脚本练习 15
10. 源码阅读路线与面试前核对清单 16
02 数据工程平台：从元数据模型到身份、注册与 Agent 18
1. 先建立项目地图，避免一上来背技术名 18
2. 必须解释准确的六个元数据概念 18
3. 沿一条注册请求掌握真实实现 19
4. 查询一致性与修改约束 19
5. 登录、凭证与授权跨三层衔接 20
6. Data Agent 的读写权限边界 21
7. 十组分级深挖问答 22
8. 两个白板练习 24
9. 源码阅读路线与个人证据 24
03 推理网关：路由决策、Redis 状态与 callback 正确性 25
1. 大项目、LiteLLM 与个人职责 25

2. 按函数体记住真实路由树 25
3. 状态刷新为什么比手写模型列表可靠 26
4. Redis 物理键、花括号与两类 TTL 26
5. 计数漂移、取消与周期校准 27
6. 指标翻倍的真正根因 27
7. 算法一与算法二：生产协作与理论必须分开 28
8. 十组深挖问答 28
9. 两个手算与排查练习 30
10. 源码路线、证据与口头禁区 31
04 路由 Simulator 的建模与工程验证 32
1 核心问题与个人工作 32
2 数据模型与执行主线 32
3 开环与闭环怎样回答不同问题 33
4 经验后端真正知道什么 33
5 状态与缓存近似的边界 34
6 指标和归档必须讲到分母与单位 34
7 分级问题与继续追问 34
8 两个可自行完成的练习 37
9 源码阅读顺序 38
第五章：编码 Agent 平台——把协作、执行与交付讲透 39
1. 先用一分钟定位项目与自己 39
2. 先分清四层对象 39
3. Node 事件循环：为什么两个 worker 可以交错，但同步认领仍然先到先得 40
4. TeamSession：从创建到完成的完整生命周期 40
5. 看板事件、重放与幂等 41
6. 增量游标与等待：怎样既少发上下文，又不漏事 41
7. 文件 apply：哈希保护究竟保护什么 42
8. 停滞检测：观测进度与判定正确性分开 43
9. Harness²：k、2k、k 和最终选择池 43
10. Python—Node 契约与配置生效 43
11. 输入准备、命名空间与产物识别 44

12. diff、adopt、rollback、pull 各解决什么 44
13. 分级追问：先回答主问题，再承受两层深入 45
14. 两个无需调用模型的练习 47
15. 本地阅读路线、实测证据与最薄弱处 49
06 跨项目追问与闭卷模拟 50
1 最容易混淆的六组概念 50
2 八组跨项目追问 51
3 十二个快速自检 53
4 一小时闭卷模拟 53
5 面试前最后核对 54

00 阅读方法与项目地图

这份手册用于陶壮的项目面试准备，配套《大模型基础设施与 Agent 工程》Beamer。目标是能独立解释项目背景、画出执行链路、说明自己的设计和实现，并在继续追问时区分代码事实、理论结论和待验证部分。它包含本机源码阅读路径与诊断记录，作为个人学习资料保存。

1 先建立完整项目地图

大项目或方向	本文讨论的具体工作	先学清楚的关系
OmniInfer 推理框架	OmniPlacement 专家均衡及模型和框架适配	OmniAttn、OmniProxy 与专家均衡是团队框架中的不同能力
Omni-EPLB 独立 SDK	Python 与 C++接入、统计、候选、权重及映射更新	它延续相关能力，不代表替代全部 OmniInfer，也不是另造一套同名原创算法
企业数据工程平台	DataHub 扩展、注册 SDK、身份权限、页面与 Data Agent	DataHub 是上游元数据底座；产线、任务与 Runtime 是团队平台背景
Omni-ELB 的 LiteLLM 路径	协作路由、独立状态刷新和指标修复	应用入口、网关选集群、后端内部 P/D 执行各有职责
路由 simulator	真实插件、请求事件、服务耗时后端与实验	它在生产链路外；与本仓库 llm_repro 模型不同
编码 Agent 平台	Delm 双 worker 适配与 Harness² 执行接入	团队平台、上游方法和个人工程改造分别归因

每个项目都先准备三句话：谁在用，遇到什么问题，我具体负责哪一段。例如“用户通过网关访问多个模型后端，热点会话会造成缓存和负载之间的矛盾；我与同事设计路由，并实现状态刷新和计量修复”。先让面试官听懂对象，再展开工具名。

2 阅读时识别四种证据状态

【已实现】指所读源码存在相应实现或可追溯历史贡献，不自动代表本机已经跑过全部测试、所有失败场景均有保证，也不代表代码全部由个人独立完成。

【理论/复现】指数学模型、性质分析或合成输入下的模拟。必须说明假设、目标、计量单位和实验范围。理论式成立不代表实际部署已经逐项实现该式。

【通用知识】是理解项目所需的计算机原理。例如进程内锁不能协调两个服务副本、调用 async 函数会返回可等待对象、服务时间方差影响排队。这些知识可以用于解释设计，不应当作自己的专有发明。

【改进建议】是可以继续做的方案。回答时使用“如果继续改，我会……”并给出验证方法，不能把它说成当前已经上线。源码核查中发现的风险会明确写成“当前快照的风险和测试缺口”，不靠旧注释掩盖。

3 分三轮学

第一轮读每章开头、执行主线和个人职责，不看代码先画图。能用两分钟讲清项目背景、个人模块、输入输出和主要取舍，再进入下一轮。建议先学推理，再学网关，按岗位重点提前读 Agent。

第二轮沿一个 token、注册请求、网关请求、完成事件或看板任务追代码，标注“谁调用、读写什么、失败如何返回”。遇到异步、进程、语言、设备或存储边界，追问所有权和完成条件。

第三轮闭卷答题，每题先用 60 秒回答再对照答案。除正常路径，还要能解释状态过期、取消、并发、重试、版本和测试缺口。每章至少做一个练习，并解释答案为何成立。

面试时先说结论，再给例子，最后说明边界。参考回答提供论证骨架，不必逐字背诵。

4 每个项目都应能回答的八件事

1. 项目目标用户、输入和最终输出分别是什么。
2. 哪些是团队或上游能力，哪些是我直接实现或共同设计的。
3. 一次操作怎样穿过模块，状态在哪些边界发生变化。
4. 核心算法的目标和约束是什么，工程实现用了哪些近似。
5. 为什么选择当前方案，与一个现实替代方案有什么取舍。
6. 哪个不变量最重要，代码如何维护，什么故障会破坏它。
7. 哪些验证已完成，哪些只是设计建议，指标到底怎样算。
8. 如果重做一次，我会先改哪件事，用什么证据判断改对了。

5 技术栈应该与具体问题绑定

不要只背“我会 C++、Python、Java、Redis、Node”。用项目解释：C++/Python 绑定服务于设备权重和高频统计；FastAPI/Pydantic 组织元数据服务边界；Java/GMS/Play 解决身份和凭证衔接；Redis/Prometheus 连接路由决策与观测；Node/EventEmitter/MCP 组织协作执行；文件哈希帮助判断输入、改动与覆盖冲突。

区分两个容易说错的名字。DataHub 中的 MCP 通常指 MetadataChangeProposal，即元数据变更提案；Agent 工具中的 MCP 指 Model Context Protocol。说“我用了 MCP”必须先讲在哪个项目、是哪一种。数据平台的个人前端实践包括 React/TypeScript；编码 Agent 平台的团队前端底座是原生 JavaScript/CodeMirror，不能串成同一份个人前端经历。

6 如何使用本地证据

入口先看面试 Beamer 和讲述提示。需要全景时看工程技术报告，需要当前调用链细节时，以本手册各章标明的源快照和函数为准。旧 README、注释、历史总结可能与当前函数体不同。

使用只读 `git show`、小例子和既有测试理解边界，无须切换证据分支或调用真实模型。本地链接依赖当前工作区，源项目不随手册打包；练习不用真实业务轨迹或凭证。

个人提交支撑具体实现，论文消融支撑团队系统条件下的组件结果，合成模拟支撑模型内现象；三种证据不能互换。内部验收与公开模拟也须分别表述。

01　推理系统：OmniInfer、OmniPlacement 与 Omni-EPLB

私人面试学习稿，2026-11-02。重点是能够从问题解释到接口、状态和失败场景，而非背诵模块名称。本章按现有材料和本地源码核对；没有重新执行设备性能实验。

标签含义：**【已实现】**为所读版本存在的代码或历史贡献；**【理论/复现】**为研究分析或合成模拟；**【通用知识】**为帮助理解系统的原理；**【改进建议】**为待验证或待开发方向。标签不代表该段全部由本人独立完成。

1. 先把三个名字、项目目标和个人边界讲明白

【已实现】OmniInfer 是总体推理加速框架。它基于推理引擎组合多种能力：OmniAttn 处理稀疏注意力与 KV 缓存模式，OmniProxy 在服务层组织预填充/解码请求，OmniPlacement 处理 MoE 专家排布与运行时负载均衡。论文以 vLLM 为实现底座，代码仓另有 SGLang 等适配。它不是我独立完成的一套框架。

OmniPlacement 是一项**能力及其实现模块**。它保留门控网络选出的逻辑专家，通过改变专家的副本数、物理位置和执行映射，缓解热点设备。历史实现位于 OmniInfer 的 `omni/accelerators/placement/`。Omni-EPLB 是该相关能力独立演进的 **SDK/仓库**，核心 Python 包仍叫 `omni_placement`；独立交付不意味着重新发明了一套算法，也不意味着代替整个 OmniInfer。

我的核心工作是专家均衡的算法工程、动态运行时、Pattern 工具、配置、测试和多模型/框架适配。现有 Git 还支持介绍 Qwen3-Next 配置、线性注意力状态形状、MLA dummy-run 的局部修复；这些不能扩写成本人主导 OmniAttn 或 OmniProxy。

面试开场可以这样说：

> 我参与的是面向大模型服务的推理加速框架，重点负责其中的 MoE 专家均衡。工作既包括副本和布局决策，也包括激活统计、Python/C++ 接口、权重更新及部署测试。相关能力后来独立演进为 Omni-EPLB SDK。我还参与了一些模型配置和框架兼容修复；整体系统和论文是团队成果。

随后用一张心中结构图定位：**请求 → `OmniProxy` → `P/D` 推理引擎 → 模型算子与设备**；**OmniAttn 与 OmniPlacement 是引擎侧的并列能力**，不能画成请求必须依次经过的两台服务。Omni-ELB 的网关路由是另一项目，不等于这里的 OmniProxy。

2. 理解实现前必须掌握的概念

2.1 MoE、专家并行和副本不是三个同义词

【通用知识】MoE 的门控网络对 token 打分，只选择部分专家计算，例如每个 token 选 Top-K 个。模型仍保存许多专家权重，只是单个 token 不执行全部专家。专家激活量不是专家数量：两张卡各放相同数量的专家，收到的 token 数仍可能差很多。

专家并行 EP 把专家权重分散到多个设备。一个 token 的目标专家不在本卡时，要把输入发送过去，完成后再把结果合并回来。副本则是同一逻辑专家的相同权重实例；把 token 分给不同副本，并不改变模型选了哪一个逻辑专家。副本占用额外内存，还需要正确加载、迁移和映射。

【通用知识】TP 是把同一算子的张量计算拆开，DP 是复制服务/计算组处理不同数据，EP 是按专家分工。一个部署可以组合三者。面试时先说明 rank 指哪个通信组中的编号，不能把 EP rank、节点、卡和

服务实例混为一谈。world_size 也必须结合接口解释；代码还有 hccl_comm_world_size，共享专家等场景下不能假设两者恒等。

2.2 Prefill/Decode、注意力成本和专家成本

【通用知识】Prefill 处理输入序列并建立缓存，Decode 利用已有上下文逐步生成。两阶段资源需求和调度方式不同，P/D 分离允许分别配置资源，但引入了 KV 状态传递与阶段协调。OmniPlacement 关注 MoE 专家执行的热点；它不能单独解决全部注意力、KV、网络或网关排队问题。

单层关键路径可粗略写为 **最慢设备计算时间 + 不可重叠的通信/同步时间**。这里的“加”是解释近似，不能据此认定通信与计算从不重叠。均衡激活量只改善计算代理；若瓶颈已转移到通信，或者迁移本身引发停顿，端到端收益可能不再随代理改善。

2.3 三种标识与两类映射

【已实现】需要同时区分逻辑专家 ID、全局物理位置、当前 rank 的本地槽位。Pattern 是 rank × layer × expert 的部署存在矩阵；执行时还要有“逻辑专家到物理副本列表/selector”和“物理槽位到逻辑专家”映射。前者服务 token 调度，后者服务权重管理与统计还原。它们表达的是不同方向，不能随意拿一个下标访问另一个张量。

3. 沿一条请求和一个 MoE 层走完执行链

【通用知识】请求先由服务调度进入一个推理后端；在引擎内部进行 tokenization、prefill 或 decode。下面只展开其中一个 MoE 层，不把整个服务路径都归为 Placement。

1. 门控网络产生 token_expert_ids 和对应分数，常见形状为 [num_tokens, top_k]。这是模型语义选择。
2. 【已实现】调用方取得当前层映射，再调用 OmniPlanner.plan()。它根据配置转到普通或 zero-expert 适配路径。plan() 使用**传入的** expert_mapping，不是自己在函数内重新读取 selector。
3. 普通路径用映射把逻辑专家转为物理执行位置；冗余场景通过相应轮转/索引逻辑分摊到副本。函数仍返回原 token 和原分数，重点修改的是执行位置 ID。
4. 【通用知识】框架按这些位置做 dispatch，远端 token 经过通信到目标 rank，执行本地专家算子，再 combine 并按门控权重合并。具体融合算子和通信组织随框架变化，不能声称所有版本都使用一份完全相同的内核。
5. 【已实现】框架把物理专家位置上的 token 计数交给 record_activation()。它在采集或动态功能开启时更新设备计数张量，并按上限回绕；这不是把完整请求内容写入日志。
6. 后台采集、优化和准备权重。主线程在接入的安全点调用 place_experts()，内部检查缓冲就绪后应用更新。下一轮执行看到新的 selector。

zero expert 的意义是：某些模型用特殊 ID 表达不等同普通路由专家的路径，不能把所有整数 ID 一律映射为普通槽位。实现用 mask 保留特殊 ID；但 mask 与索引算子的求值、图编译兼容仍需测试，不能仅凭 where 一词认定所有非法下标都会被惰性跳过。

4. 数学对象、贪心结论与复杂度边界

4.1 先分副本，再决定它们放在哪里

【理论/复现】对一层设 E 个专家、N 个设备、固定工作量 a_e ≥ 0；r_e 是专家总副本数，u_e 是其上限，B 是额外副本预算。副本子问题是

$$\min_{r}\max_{e} a_e/r_e, \quad 1 \le r_e \le u_e,\ r_e \in \mathbb{Z},\ \sum_e r_e \le E + B.$$

这假设同一专家的工作能在副本间均匀分担、各副本成本相同，且暂不考虑设备组合。静态 Python 函数以最大堆维护 a_e/r_e，每次给最大且未达上限者加一个副本，再更新优先级。

为什么这个子问题有严格结论？给定目标阈值 t，专家至少需要 max(1, ceil(a_e/t)) 个副本；若超过其上限，或总需求超过预算，t 不可行。只要还存在高于 t 的可增副本专家，堆就先处理它，而不会把预算花到已低于 t 的专家。因此，对于一个预算内可行的阈值，贪心不会因把资源给无关专家而错过它。由阈值可行性可说明：**上述模型下，最大单副本负载达到最优值**。这不是所有布局和时延的全局最优证明。

【已实现】原函数允许 log1p 变换。一旦开启，堆优化的是变换后的工作量，不能把所得最优值直接称为原 token 负载最优。还要辨明参数口径：数学的 u_e 是总副本上限；静态函数的 expert_redundant_limit 表示额外副本上限，代码用 1 + limit。配置、C++ 构造参数与数学符号应逐层核对。

设备放置另设 x_de ∈ {0,1}，满足 Σ_d x_de=r_e、Σ_e x_de≤C_d，设备代理负载为 L_d=Σ_e x_de a_e/r_e，目标为 min max_d L_d。同一专家不能在同卡重复放置；总副本数、容量和专家覆盖都要检查。原放置函数把 M 个副本实例按负载降序处理，每次放到可行的最低负载设备。顺序会影响结果，找不到合法位置时会显式失败。

4.2 一个足够在白板上说明边界的反例

【理论/复现】六个不同专家的负载为 [8,7,6,5,4,0]，两设备、每设备三槽、每专家一份。降序最轻设备贪心可得到 A=[8,5,4]、B=[7,6,0]，最大负载 17；合法布局 A=[8,7,0]、B=[6,5,4] 的最大负载 15，而总负载 30 决定下界至少 15。副本选择在本例没有困难，设备组合仍可使贪心不最优。

因此回答“最优吗”要先问“哪个目标、哪些条件”。不能把单副本最优偷换成设备最优，也不能把设备代理负载较小偷换成端到端性能必然更好。原 Python 的主机交错重排还使用具体分组假设，不是任意拓扑和链路容量的联合最优求解器。

【已实现：当前静态 Python 脚本】distribute_experts_to_ranks() 的重排部分固定 num_ranks_per_host=16。以 32 个 rank、两主机为例，循环按 new_host_id=i%2 选择主机，再用该主机的局部计数决定目标行。因此**原行 0、1、2、3 分别写入新行 0、16、1、17**，不是反向把新行 0、1、2、3 从这些原行读取。若全局 rank 按每主机 16 个连续编号，原本相邻的设备布局就会交替落在两主机上。例如原行 0、1 上的两个较高负载布局会被分到新 rank0 和 16。这里置换的是整行专家布局，不改变各行本身的专家组合；它意在跨主机交错分布，但没有显式求解链路带宽、通信流量或任意网络拓扑。本章也没有独立实验验证该重排的性能收益，不能把示意作用当成已测量的改善。

4.3 复杂度必须拆开计算

阶段	时间	主要空间
堆式副本分配	O(E+B log E)，heapify 加最多 B 次更新	O(E)
原静态贪心放置	O(M log M+MN+NE)	O(M+NE)
动态候选搜索	候选数乘每次生成/评分成本，另加统计和指令开销	取决于候选保存和布局规模

放置复杂度中的 M 是总副本数，MN 来自逐副本扫描 N 设备，NE 来自矩阵初始化/复制等。多层、多个预算和拓扑处理另计。这里没有把 Python 静态函数的复杂度直接当成整个 C++ 在线优化器的复杂度。

5. 动态路径：从物理计数到候选和安全切换

5.1 统计必须知道计数属于哪个布局版本

【已实现】`ClusterActivation.collect()` 汇集设备计数并做本轮差分。若计数上限为 C，当前值小于上次值时补 C 再相减。该方法隐含相邻采样间没有发生多次不可辨识回绕；采样过慢、进程重启或计数器被重置，都需要额外处理。

`updateShiftWindows()` 按当前“槽位→逻辑专家”映射，把多个物理副本的增量汇总为逻辑专家激活。若专家 A 两个副本各接收 15、12 个 token，总需求是 27，不能把它们看成两个独立中等热点。布局改变后，旧槽位上的增量必须按旧映射解释，不能拿新映射给旧计数贴标签。

本地快照的关键细节：源码虽然维护逻辑专家环形历史，当前 `PlacementOptimizer.extract_input_data()` 传给候选优化器的主要输入仍是 `getExpertActivationCount()` 取出的本轮物理 delta。不能看到 `updateShiftWindows` 就说实际决策必然使用“十窗口平均预测”。Python 构造处传入窗口参数 10，而 `ExpertActivation` 的环长在头文件中为 20；要继续追踪 getter 和调用链，区分维护了什么统计、决策真正读了什么。

离线 Pattern 的 Step1 又是另一口径：它按采集 rank/position 累计，并存在 `ceil(count/128)` 量化，不读取当前 Pattern 来自动还原任意冗余布局。因此离线输入必须核对编号对应关系，CSV 也不能不加解释地称为原始 token 总数。

5.2 门禁判断的是布局代理，不是已测量的净时间收益

【已实现：仅指本章核对的 Omni-EPLB `bb5122d`】评分逐层计算，不把不同层或不同请求的比值再求平均。当前布局将该层每个 rank 的有效物理槽位激活增量求和，得到 `L_d`；`compute_placement_ratio_combined(..., type=0)` 返回

$$R_{old} = input_ratio = \frac{\max_d L_d}{\sum_d L_d / N}.$$

候选评分先按旧布局汇总各逻辑专家工作量 `a_e`，再统计**候选布局**中的副本数 `r_e`，假设副本均匀分担，得到 `L'_d=Σ_e x_de a_e/r_e`。`simulate_placement_ratio()` 返回 `R_new=max_d L'_d/(Σ_d L'_d/N)`，输出日志的 after ratio 也来自这一代理计算，不是迁移后的实测时延。比值越小越好，均衡时

为 1。两个评分函数都规定：平均负载为 0 且最大负载也为 0 时返回 1，否则返回正无穷；在正常的非负激活输入下，全零层因此按均衡处理。

候选路径先判断当前比值，`R_old < input_ratio_threshold` 时保持原布局，否则按 rank 数步长枚举冗余预算并评分。只有严格满足 `R_new < R_old − δ` 才接受，而不是改善恰好等于裕度也通过。小例子：当前负载 [12,8] 的平均值 10、比值 1.2，候选 [11,9] 的比值 1.1，δ=0.05 时通过。再用二进制浮点可精确表示的边界例：R_old=1.5、R_new=1.25、δ=0.25，因 1.25 < 1.25 不成立而拒绝。0.05 是比值的绝对差，不是相对改善 5%，也不是性能收益；头文件默认值只用于理解该快照，不代表所有版本和部署都使用相同参数。

另一个 high_low_ratio 调用 type=1，计算最大负载除以截到至少 1 的最小负载，用于不均衡告警；其零均值分支同上。它与用于候选排序和接受的 max/avg 是不同指标，不应混用。

【理论/复现】若用时间单位建模，可考虑 **热点持续轮数×每轮节省 > 不可重叠复制和切换成本**。这解释短热点为什么可能不值得搬，但当前 ratio 门禁不等于已经估计了所有这些成本。本仓库的合成动态模型有单独的窗口与复制时间假设，不是设备运行时的逐指令等价复刻。

5.3 目标布局先变为可执行的指令

【已实现】PlacementOptimizer 比较当前 f 与目标 g，尽量保留本 rank 的交集位置，选择已有副本作为源，产生 ADD/REMOVE 等指令。执行前检查源/目标位置是否与当前专家一致。merge_instructions() 合并不冲突的层，reorder_instructions() 按 rank 冲突组织批次；一批中同一 rank 不能无限承担多路传输。

Distribution 为地址、长度、dtype、目标 rank 与接收缓冲建立描述。跨 rank 调 HcclSend/Recv，本地搬移用设备到设备拷贝。方法名叫 hccl_batch_send()，但当前实现仍遍历条目调用并逐项同步 stream；不能把“批量组织”夸成“一次底层原子批发送”或“全部传输完全并行”。

5.4 安全点究竟保护什么

【已实现】接收先落到独立设备缓冲；后台完成一轮准备并设置 buf_ready_flag_。主线程调用 do_placement_optimizer()，把就绪数据复制到最终 HBM，等待拷贝 stream 完成，更新变更层 selector，再清除标志。后台通过 round/batch 状态 AllGather 做握手，之后重新采集以隔开布局切换前后的统计。

这里尤其要分清：后台可能已更新控制侧“位置→专家”元数据，而主线程使用的 selector 尚未发布。不能说“所有映射和权重一起原子提交”。真正要检查的是，推理数据路径在可见 selector 切换前不会使用未就绪权重；控制元数据、执行映射和统计需要有一致的时序解释。

6. Python、C++、线程与设备内存怎样共同工作

6.1 语言边界也是所有权边界

【已实现】Python 负责模型配置、tensor 接口、Pattern 加载和框架编排，C++承担统计、候选、指令、权重管理和通信。setup.py 用 pybind11 构建扩展；placement_handler.py 通过 tensor 的 data_ptr()、shape、dtype 等信息把存储交给 C++包装，并把专家权重字典整理为地址描述。

【通用知识】传一个整数地址不等于复制数据或转移所有权。Python tensor 必须活到后台不再使用它，布局、stride、连续性、dtype 和字节数也必须匹配；设备异步操作完成前，不能释放或复用底层存储。看到 pybind 返回 `reference`，只能说返回借用引用，不能说自动复制出独立对象。还应检查构造引用是否由 Python 父对象长期持有；本章不把未发现的 `keep_alive` 策略当作已有保证。

6.2 GIL、互斥锁、原子标志和 stream 同步各管一层

【通用知识】GIL 约束 Python 解释器执行，不会自动保护原生 C++线程与设备 DMA。互斥锁控制主机共享对象的临界区，原子标志表达状态/发布关系，ACL stream/event 处理设备操作先后；HCCL 握手处理跨 rank 进度。这四件事不能互相替代。

【已实现】`Placement` 有 worker 线程、停止/暂停/ready 原子标志和 mutex 接口；`MoEWeights` 也有 mutex。但“类里声明了锁”不证明每条读写路径都加锁。`do_placement_optimizer()`主要依赖就绪协议；必须沿实际调用核对数据竞争，不能靠字段名做线程安全结论。

后台设置 ready 后会轮询等待主线程清标志，再做跨 rank 握手。`stop_thread()`设置停止标志并 join，不意味着每个内部等待循环都能立刻响应退出；当前有等待路径没有在每次循环检查 stop 条件。它是需要故障注入验证的风险，不能据此承诺有界停机或完善异常回滚。

6.3 context、通信器和接收缓冲

【已实现】`enable_new_context` 允许后台设置设备并创建独立 ACL context，否则采用调用线程传入的 context；随后创建 stream 并初始化 HCCL 通信器。ACL context 是设备运行时上下文，HCCL communicator 是通信参与者关系，stream 是操作队列，三者不是同一对象。独立 context 有助于降低生命周期耦合，但不能自动消除设备带宽竞争或所有跨流依赖。

接收缓冲按专家大小与队列容量预分配，描述队列记录最终目标地址。必须验证每专家多块权重的长度、偏移、接收空间上限与源/目标 dtype。批次传完后才复制最终权重，是减少错误可见性的办法；若框架安全点挂错位置、某 rank 漏执行或者进程退出，协议仍可能停滞。

【改进建议】优先补布局版本号、可取消的等待与超时传播、明确的所有权约束、按阶段故障注入和跨 stream 事件验证。它们应作为下一步设计讨论，不说成当前已交付。

7. 本人具体做过什么，证据如何落到代码

【已实现】以下锚点能支撑具体工作，不以提交数量证明贡献大小：

提交	可以讲的设计/工程工作	不应扩大为
113661c5、21fcc9ed、4915d693、a9636ef9	Pattern 流水线、非均匀冗余与工具迭代	整个推理框架原创
0c518500、90ccecdb、e38bac0f	共享/zero expert 及图编译适配	改变了模型 gate 训练方法
82fb0436	批量传输、动态/静态相关运行时迭代	生产零停顿、通信完全重叠
8f303d57	C++/Python 测试与构建流水线	所有硬件/模型已穷尽验证

提交	可以讲的设计/工程工作	不应扩大为
16a702c1、aeca07d3、b10a58a9	配置加载、CLI 及框架接入修复	主导所有框架 adapter
1d4f4596 → 92e2bce	新通信 context 能力从旧仓到独立 SDK 延续	两项独立原创算法
ef875720	causal_conv 初始/最终 state 转置修复	OmniAttn 算法设计
f2f18502、63912ce6	Qwen3-Next 配置、MLA dummy-run 调用修复	所有模型端到端实现

讲外围修复时说清触发：例如卷积初始 state 与输入拼接前布局不一致，修复相关 transpose，并保持最终 state 接口形状一致。只能据补丁说明修复对象，不补造事故规模、业务损失或未保存的性能提升。

8. 十二组分级追问：回答后还能再往下讲两层

Q1　基础：这三个项目名为什么要分开？

回答：框架、能力、独立 SDK 是不同层次。我的主线是专家均衡，其实现经历框架内集成和独立演进，不能把整个框架都归到个人。

追问一：独立后需要什么契约？Pattern/模型维度、权重与计数存储、逻辑到物理映射、统计钩子和安全点调用。

追问二：是不是直接插入任意引擎就能用？不能。当前有设备工具链、模型语义和框架生命周期依赖，需逐项适配与验证。

Q2　基础：为什么不用改 gate 解决热点？

回答：本方案保持 gate 的逻辑选择，优化它选中专家的执行资源；改 gate 可能涉及模型语义和质量，属于另一类取舍。

追问一：副本选择会影响数值吗？理想上相同权重执行同一函数，但 dtype、算子和合并顺序仍需数值回归。

追问二：零专家为什么特殊？特殊 ID 不一定对应普通权重槽，mask、索引和编译路径必须匹配模型定义。

Q3　基础：卡上专家数一样为什么还慢？

回答：负载是各专家被激活的工作量之和；专家数只描述容量占用，不描述 token 分布，最忙设备可能决定同步进度。

追问一：token 数就是时间吗？不是，它是同类专家条件下的代理；算子、批形状、通信会改变单位成本。

追问二：如何证明瓶颈转移？联合看设备/层耗时、通信等待与业务时延，比较均衡前后关键路径，不能只看热图。

Q4　进阶：你说的贪心到底哪里最优？

回答：固定工作量、均匀分担、等副本成本和整数上限下，堆分配解最大单副本压力子问题，阈值需求可直接检验。

追问一：放置也最优吗？不是，六专家反例能说明最低负载贪心仍输给另一合法组合。

追问二：副本大小不同怎么办？等成本条件消失，应显式加内存/资源预算，原证明不能原样使用。

Q5　进阶：为什么不能把复杂度只写成 log E？

回答：log E 仅是一次堆操作，完整副本分配有初始化与 B 次更新；放置还排序、扫描设备并维护矩阵。

追问一：多层会怎样？按各层规模求和，预算搜索再乘候选次数，不凭一个子函数替代全链路。

追问二：为什么不全部换成堆？可行性还依赖同卡唯一性、容量和拓扑，单一最小负载堆未必支持快速合法选择。

Q6　进阶：统计为何要先物理后逻辑？

回答：执行实际发生在物理槽，多个副本合计才是逻辑专家需求。回绕差分要与当时布局对应。

追问一：旧 delta 套新布局会怎样？把旧槽位 A 的工作错记到新槽位 B，下一次决策可能追着假热点搬。

追问二：窗口预测具体怎么实现？当前快照维护历史，但主要候选输入读本轮 delta；不能把历史存在等同于均值驱动决策。

Q7　进阶：为什么要两道门槛？

回答：触发门槛避免本来均衡时搜索；改善裕度避免小代理收益引发搬迁，减少无效更新与抖动。

追问一：能保证赚回迁移成本吗？不能，当前 ratio 门槛没有完整计入热点持续期与不可重叠时间。

追问二：怎么改？用同一时间单位估计收益、迁移和不确定性，并在留出负载与真实系统中检验。

Q8　进阶：权重迁移为什么不直接覆盖？

回答：推理可能仍用原权重；先接收缓冲，再在安全点复制并更新 selector，避免提前发布未就绪数据。

追问一：ready 为真就代表设备可见吗？还要确认相关 stream 完成及跨流先后；主机原子不替代设备同步。

追问二：中途失败会完整回滚吗？当前不能据代码宣称事务回滚；需要分阶段列故障点和恢复状态。

Q9　深入：批量传输优化了什么？

回答：把差异指令按层/轮次与 rank 冲突整理，统一准备权重描述与缓冲，限制批内资源冲突。

追问一：是不是一个 HCCL 调用发完？当前仍逐条 Send/Recv 并同步，方法名中的 batch 不等于底层单次原子批操作。

追问二：还能怎样减少同步？先确认依赖图与缓冲上限，再讨论事件或组批；不能先删同步再盲测是否出错。

Q10　深入：Python 传地址给 C++最大的风险是什么？

回答：存储生命周期与布局不匹配。地址不拥有 tensor，后台线程可能晚于 Python 局部对象结束才使用它。

追问一：GIL 能保护吗？不能保护设备 DMA 和原生线程；需显式所有权与同步协议。

追问二：怎么审查？从 data_ptr 来源追到释放方，检查 shape/dtype/byte size、持有对象与最后一次异步使用。

Q11　深入：独立 context 与线程安全有什么关系？

回答：独立 ACL context 让后台设备运行时关系更明确，但它不等于新通信组，更不自动保证共享对象线程安全。

追问一：声明 mutex 够吗？不够，要看每次访问是否遵守同一锁/发布协议。

追问二：停止标志设置后为何还会卡？join 等线程退出；内部轮询或 collective 若不响应取消，仍可能无限等待。

Q12　综合：你如何证明自己做了什么、实现有效？

回答：先落到个人提交和模块，再用约束/映射/状态测试验证正确性，最后讨论设备与系统实验。团队论文不能替代个人归因。

追问一：能引用论文性能吗？可以按公开配置与消融比较说明团队组件效果，须同时保留 TTFT 等权衡，不能宣称个人独立收益。

追问二：合成模拟能证明什么？能检验给定假设下的机制和退化条件，不能自动预测真实设备数值、证明全模型性能或还原生产结论。

9. 两个纸笔或小脚本练习

练习一：算副本、找反例、核对最优对象

【理论/复现】先对 [12,8,4,4]、B=2、每专家上限 2 手算堆操作。答案是副本 [2,2,1,1]，单副本瓶颈 12 降为 6。再设两设备、每设备三槽，构造各含 e1(6)+e2(4) 再分别加一个 4 的合法布局，设备负载均为 14。最后用下面的独立小脚本验证六专家的放置反例；不要导入设备 SDK。

```
from itertools import combinations

loads = [8, 7, 6, 5, 4, 0]
groups = [[], []]
totals = [0, 0]
for idx in sorted(range(6), key=lambda i: -loads[i]):
    d = min((d for d in range(2) if len(groups[d]) < 3),
            key=lambda d: (totals[d], d))
    groups[d].append(idx)
    totals[d] += loads[idx]
best = min(max(sum(loads[i] for i in a),
```

```
                sum(loads[i] for i in range(6) if i not in a))
            for a in combinations(range(6), 3))
print(totals, best)  # [17, 13] 15
assert max(totals) == 17 and best == 15
```

做完应能回答：为什么副本子问题的阈值证明没有被这个例子推翻？因为例子反驳的是后续设备组合贪心。再问：负载全乘同一常数，布局排序是否改变；开启 log 变换后，还能否把结果称为原负载的最优？

练习二：用四个槽位演练统计版本和安全点

【理论/复现】旧布局为 [A,B,A,C]；计数上限 100，前值[90,20,60,10]，当前值[5,28,72,15]。手算 delta 为[15,8,12,5]，逻辑负载 A=27、B=8、C=5。目标布局交换 B/C，变为[A,C,A,B]。若用新映射解释旧 delta，会错成 B=5、C=8。

```
from collections import defaultdict

before, now = [90, 20, 60, 10], [5, 28, 72, 15]
delta = [(c - p) % 100 for p, c in zip(before, now)]
def aggregate(mapping):
    ans = defaultdict(int)
    for expert, count in zip(mapping, delta):
        ans[expert] += count
    return dict(ans)
print(delta, aggregate('ABAC'), aggregate('ACAB'))
assert aggregate('ABAC') == {'A': 27, 'B': 8, 'C': 5}
```

在纸上加四列：控制侧位置元数据、接收缓冲、最终权重、推理 selector。先写“接收中”、再写“ready”、再写“主线程拷贝完成”、最后写“selector 发布”。分别注入：某 rank 不再握手、主线程一直不调用安全点、Python 提前释放计数 tensor。说明哪些状态不能前进、当前有哪些保护、还缺什么取消/所有权机制。这是协议练习，不是对现有代码做过故障注入的声明。

10. 源码阅读路线与面试前核对清单

本章阅读的 OmniInfer HEAD 为 c92da53873c19c75e8df4d7924ef04b2fcc99d0c，Omni-EPLB HEAD 为 bb5122d117e9a63ecdf555240dba79c75be74393。HEAD 只定位所读版本；当前文件不等于每行均为本人作品。

1. **建立项目地图。**读 OmniInfer README、OmniProxy 说明；论文看本地 OmniInfer v2 的三组件定义与系统实验边界。
2. **看 SDK 接入而非先钻 C++。**读 Omni-EPLB README、Planner、ExpertMapping、Python/C++桥接。画出对象所有者与 data_ptr 传递。
3. **读静态子问题。**从 Pattern Step2 的两个函数到 Step1/3，标注预算、量化与覆盖检查各自口径；结合复现模型说明。
4. **追动态数据。**从 激活统计 到 Optimizer、LoadBalancer，明确本轮 delta 和历史统计哪一个进入哪个决策。
5. **追一次更新。**读 Placement 管理器、Distribution、MoEWeights；给每一步标主线程/后台、Host/Device、同步/异步、旧/新布局。

6. **用测试检验理解。**看 omni_placement/cpp/test/cpu/、test/npu/ 与 tests/run_test.sh。已有 mock 证明的是所模拟边界，NPU 存在才执行的分支不能说成当前机器已经通过。

【已实现】可对照工程技术报告理解全景，但实现细节以本章标出的具体版本调用链为准；指南中的旧目录、示例硬件、参数、注释可能滞后。若需要重查个人补丁，执行 git -C <证据仓库> show <本章提交>，只读即可，不切换或重写证据仓库。

面试前应能不看稿完成四件事：画出三个名称关系；从一个 token 走到物理专家并解释统计；白板说明贪心成立范围及反例；逐阶段解释一次权重更新及两种失败。能够坦诚区分“我做过”“团队具备”“理论上成立”和“下一步想改”，比背诵未经核验的性能数字更能承受深挖。

02 数据工程平台：从元数据模型到身份、注册与 Agent

私人学习材料，2026-11-02。对应面试稿第 11—14 页。依据本地 `data_infra_platform` 快照 `36ffaff8`、主计划及实际函数体；只解释实现和设计，不包含服务地址、凭证、员工编号或业务规模。

状态标签含义：【已实现】有本地代码或提交依据；【理论/复现】报告模型或独立验证；【通用知识】用于解释原理；【改进建议】尚不能表述为交付成果。

1. 先建立项目地图，避免一上来背技术名

【已实现】团队项目是数据工程平台，用户包括数据工程师、产线开发者和资产治理人员。整体涉及资产接入、产线版本、生产任务、Runtime 执行、元数据管理与统一门户。DataHub 是其中的开源元数据底座，提供实体、属性、搜索、Schema 和血缘；它不是整个数据工程平台。

我的工作集中在元数据与治理：Python 的 Extensions API/注册 SDK；Java GMS 的企业身份、准入、角色与凭证；Play 服务端的会话和代理衔接；React/TypeScript 的资产、权限和注册页面；Data Agent 的工具交互。任务执行、制品版本等消费我的接口，不代表这些模块全部由我开发。

可以用一分钟讲清：

> 数据工程平台要回答“什么数据，用哪版产线，由谁执行，得到什么结果”。团队负责完整生产流程，我重点建设元数据接入和治理。基于 DataHub，我把注册逻辑拆成可复用 SDK 与 API，接入企业身份和领域权限，并打通前端与元数据 Agent。技术难点是让身份、实体约束、写入和查询语义跨 Python、Java 与前端保持一致。

【通用知识】把项目理解成四个域更容易深挖：身份域回答“谁”；治理域回答“可做什么”；资产域回答“操作什么对象”；生产域回答“如何执行并回写”。SSO 解决身份，RBAC/领域策略解决授权，二者不能互相替代。

2. 必须解释准确的六个元数据概念

【已实现】Entity 是一个可独立标识的对象，如数据集、产线和任务。Aspect 是围绕实体的一组结构化属性，例如注册信息、Schema、归属或运行属性。实体不是把所有字段塞在一个无边界 JSON；Aspect 允许分域维护和演进，但多个 Aspect 的更新不天然形成一个整体事务。

URN 是实体身份。稳定的同一身份让重复上报、查询和血缘引用指向同一对象。数据集名称、平台和环境可能参与 URN。本 SDK 的 Task 映射为 DataJob，URN 包含所属 DataFlow 与 name；run_id 是独立属性，不自动进入 URN，也不是必填身份字段。不能只靠显示名判定两个实体相同，也不能在改名时忽略旧引用。构造规则由 `MetadataEntity.build_urn()` 和类型定义落实。

这里有两个不同的 MCP：

缩写	在哪里出现	实际含义
Metadata Change Proposal	注册 SDK 的 build_mcp_list()	提议修改一个实体的某个 Aspect 的元数据事件。

缩写	在哪里出现	实际含义
Model Context Protocol	Agent 工具生态与工具适配命名	模型接入工具/资源的一类协议；与元数据变更提案不是一回事。

【已实现】本项目 Agent 代码有本地工具注册表和 `MCPToolAdapter`。不能凭类名就声称已经运行完整远程协议服务，必须继续检查调用链和传输。Emitter 是元数据写入适配；GraphQL 是服务查询接口；Elasticsearch/ES 是搜索索引。GraphQL 不是数据库，“走 GraphQL”也不是强一致性的通用保证，要看具体 resolver 读取什么。

3. 沿一条注册请求掌握真实实现

【已实现】HTTP 路由解析身份和请求，也可以在可信进程中直接使用 SDK。核心链路如下：

1. `register()` 校验实体类型，解析 YAML/字典并合并参数，迁移旧字段。
2. 从可信 actor 或凭证反查主体，覆盖用户填写的 creator。不能允许调用者把自己写成另一个注册者。
3. 根据 SDK/REST 调用模式执行权限检查；“API 先查过”是明确的信任边界，不是所有直接调用默认安全。
4. `MetadataEntity.from_dict()` 形成实体，构造 URN，填充时间戳，检查字段与 Aspect；注册路径还校验 Domain。
5. 按 URN 获取进程内注册锁，在锁内再做存在性判断。已有对象且不覆盖时返回 skipped；任务按配置检查产线、输入等依赖。
6. 构造多个 Metadata Change Proposal，经 Emitter 发布；发布失败返回明确错误。
7. `_poll_verification()` 使用 GraphQL 的 `exists_by_urn()` 轮询存在性。超时会出现“发送已完成，但验证超时”，不能当作确定没有写入。

实体层主要做建模和纯计算，服务层编排，数据访问层承担查询，发布层承担写入。原因是 URN、校验和 MCP 构造可独立测试；替换发布通道或客户端时不必重写实体规则。替代方案是集中在一个大服务类，早期开发快，但连接、权限、序列化和业务规则会相互牵连。

【通用知识】幂等、互斥和事务分别回答“重复请求结果是否可控”“并发能否同时进入”“一组修改能否原子提交”。当前按 URN 的 `threading.Lock` 只能协调同一 Python 进程；多 worker、多副本各有锁。引用计数回收锁表可避免不同 URN 越积越多，但不解决分布式竞争。

【改进建议】真正跨副本的创建唯一性应考虑服务端约束、版本条件更新、请求幂等键或带 fencing 的协调；不能只加 Redis 锁就宣称 exactly-once。进程崩溃发生在部分 Aspect 已写后，重试可能覆盖或补齐，也可能看到部分状态，需要依据写入协议分析。

4. 查询一致性与修改约束

【已实现】SDK 通过搜索发现候选实体，通过已知 URN 的 GraphQL 精确获取和检查存在。搜索索引异步更新，所以“刚注册、能精确读、搜索暂时没有”并不矛盾。当前验证轮询检查存在，不证明所有字段都达到预期版本，更不证明所有搜索条件都可检索。

amend() 先精确定位对象，再对当前所属域检查修改权限，按 merge/overwrite/append/replace 等策略合并。creator、注册身份及类型相关不可变字段要单独保护；只保护页面控件挡不住 API 请求。先查旧对象也避免调用者通过篡改请求中的 Domain 把授权判断移到另一个域。

【通用知识】列表 append 重试可能产生重复，merge 去重也不等于任意复杂对象幂等；需要知道去重键和覆盖规则。超时后的重试应先以同一 URN 查询，而不是生成新名字重复创建。替代方案是异步提交后返回任务 ID，但这需要提交状态查询和用户反馈设计。

5. 登录、凭证与授权跨三层衔接

【已实现】GMS 是服务端元数据和授权能力的重要承载处。企业 SSO 接入验证外部身份，再做准入与用户供给；角色策略决定操作权。准入名单是“能否进入”的粗门槛，Publisher/Editor 等角色和领域策略是“进入后能做什么”。React 的按钮显隐只优化体验，最终判断必须在服务端。

Play frontend 是 Java 服务端，不等于 React 浏览器代码。它处理登录回调、平台会话、后端代理和凭证衔接；React 调用对应接口展示授权、注册、凭证等页面。NeverTokenManager 为用户供给长效 PAT，在代理层注入，失败时可回退会话 token。它用每用户锁和二次检查减少同进程重复发放，读取实时 Aspect 而非依赖可能延迟的搜索列表。

GMS 的 secret Aspect 通过 SecretService 加密存储可再次取用的凭证，并校验调用者为本人或具备凭证管理权限。面试要区分加密保存、凭证哈希、访问授权和撤销：它们处理不同风险；加密不意味着管理员无条件取用，也不意味着已有客户端缓存立刻失效。

用虚构用户“小林”串起凭证与 actor

【已实现】以下示例假设启用扩展 API 的 DataHub 权限模式，经 Play 代理访问 Chat，不使用简单共享 Token 放行，也不使用关闭鉴权的可信模式。小林只是虚构用户；名称不是凭证。若走同源反向代理直达扩展 API，应另查其头与 token 的转发配置，不能默认经过同一个 Play 方法。

一跳或阶段	携带与校验什么	actor、权限与边界
登录结果到浏览器	企业认证链确认小林身份；Play 登录流程接收 GMS 访问 token，建立含 actor/token 的平台 session。	浏览器 session 的存在，不证明其中 GMS token 此刻仍有效。
浏览器访问 Play	Play 框架解析 session；Authenticator 按配置检查可转发资格，包括 Cookie/actor 一致性或 Authorization 头存在。	启用 GMS 认证时，这里不是最终 token 验签点；AuthUtils 明确将 token 有效性留给下游 GMS。React 不产生权威身份。
Play 获取或选择凭证	NeverTokenManager 用小林自己的 GMS session token 读写其 PAT；GMS 检查对应操作权。代理优先小林 PAT，回退 session token，无 session 时才透传原 Authorization。	不是 Play 拿管理员身份替小林无条件发放凭证。代理剔除客户端 actor 头，再从服务端 session 写入 actor 头；不能把任意来源的同名头视为可信。

一跳或阶段	携带与校验什么	actor、权限与边界
扩展 API 解析登录身份	require_login_with_actor() 提取 Authorization，权限客户端用该 token 向 GMS 查询本人身份和权限，可利用权限缓存。	Python 以 token 反查结果中的 username 为首选 actor；该值再用于 creator。成功登录只说明身份解析通过，不等于具有注册权。
Chat 到注册工具	路由计算 can_register：global 模式看全局角色，domain 模式看是否存在可编辑域；工具在 False 时拒绝，并把 actor 传给注册服务。	这是注册动作门禁，不能直接推导对任意目标域有写权限。actor 是操作主体信息，不会自动把下游调用切成小林的凭证。目标域约束仍需沿具体注册路径核查。
适配器访问 GMS	MCPToolAdapter 的 SDK/GraphQL 客户端使用其初始化 token；注册使用注入的 MetadataService 或适配器 token 创建的服务。	下游 GMS 按实际携带的 token 判权。因此 API 确认了小林身份，仍不证明工具的每次数据读取都以小林权限执行；读工具逐用户可见性尚待验证。

【通用知识】session、PAT、actor 和 `can_register` 分别是会话/凭证、可用于 API 的访问凭证、主体标识和动作判定结果，不能互相充当。这里的“API/PAT”是扩展 API 接受的 DataHub 访问凭证，不是模型网关的 LiteLLM API key。上述依据分别在 `Authenticator/AuthUtils`、`Application.proxy()/getAuthorizationHeaderValueToProxy()`、权限依赖、Chat 路由和 `MCPToolAdapter`；不要仅根据变量叫 user 就判断实际用了用户权限。【改进建议】长效凭证适合减少调用衔接摩擦，同时需要轮换、撤销、最小权限与缓存失效策略。不能把已实现的便利性说成完备的密钥治理。若被问多副本供给，承认本地锁范围，并进一步讨论后端唯一约束或创建冲突处理。

6. Data Agent 的读写权限边界

【已实现】当前 Chat 路由使用 `require_login_with_actor()` 登录依赖；旧 docstring 中“chat 不设鉴权门禁”的描述不能覆盖当前函数体。路由解析 actor 与 can_register，再传到工具适配器；注册工具无权时拒绝，身份解析失败时关闭注册能力。搜索、实体、血缘、Schema、Profile 与注册使用不同工具，返回实体链接供用户检查。

【已实现】会话历史当前保存在服务进程的字典中。不能把随机 session_id 当作已证明的用户隔离，也不能宣传多副本共享持久历史。读工具的逐用户数据可见性，要追踪实际传给下游的 token 与策略，不能只看 Chat 已登录就断言完成。

【通用知识】语言模型做工具选择不具有授权权威；工具参数也属于不可信输入。元数据描述中的指令不应扩大工具权限。【改进建议】逐用户 token/受控委托、工具 allowlist、资源级授权、会话所有权绑定、结果审计都可继续补强，应说“我会如何做”，不要说“项目已经全部具备”。

7. 十组分级深挖问答

Q1【基础】你做的是 DataHub 还是数据工程平台？

答：团队做完整平台，我基于 DataHub 负责元数据接入、治理和交互。上游模型、搜索与血缘能力是基础，我实现企业适配与注册框架。

第一层追问：为什么不从零写？答：复用成熟元数据模型与生态，把精力放在平台差异和接入契约。

第二层追问：如何避免深度 fork？答：扩展 API/SDK 尽量独立；必须修改的身份、角色和前端接入保持清晰补丁边界。不能声称已做到完全无侵入。

Q2【基础】Entity、Aspect 和 URN 如何协作？

答：URN 标识对象，Aspect 表达对象某一方面的属性，Entity 组织这些属性。注册把属性转换成针对该身份的变更提案。

第一层追问：显示名重复怎么办？答：身份依赖平台、环境和类型键，不按显示名裸比。

第二层追问：URN 改了呢？答：等同身份迁移，旧血缘和引用需要处理；不能只修改 UI 名称就认为完成重命名。

Q3【进阶】为什么要拆 SDK 和 API？

答：API 为浏览器或远程客户端提供身份与边界；SDK 让任务和 Runtime 等可信组件复用同一实体与编排规则，避免 HTTP 成为唯一调用方式。

第一层追问：直接 SDK 会绕鉴权吗？答：取决于 require_auth、user_token 和可信模式配置，必须明确部署边界。

第二层追问：哪个层注入 creator？答：服务从可信主体注入，强制覆盖用户字段；HTTP 路由传 actor 不等于任意客户端头都可信。

Q4【进阶】加锁以后注册就是幂等的吗？

答：不是。锁串行化竞争；稳定 URN、存在性检查及 skip/overwrite 决定重复行为。当前锁还只在进程内。

第一层追问：两个副本同时判断不存在？答：两边可能同时写；同 URN 不保证多个 Aspect 原子一致。

第二层追问：怎么改？答：把唯一性/版本判定推到可靠服务端，结合幂等键；分布式锁只是协调手段，还要考虑超时、租约和旧持有者写入。

Q5【进阶】为什么注册成功却搜不到？

答：写入确认、实体存在和搜索索引可见是不同阶段。当前轮询 GraphQL 存在性，不等于验证 ES 所有字段。

第一层追问：GraphQL 天然强一致？答：不是，要看 resolver，不能用接口名替代存储语义。

第二层追问：轮询超时怎么报？答：明确“已发送、验证超时”，保留 URN/trace_id 供查询，不盲目创建新对象。

Q6【进阶】如何防止伪造请求中的域绕过原对象授权？

答：【已实现】amend() 先按 URN 读取原对象，再以 existing_data 的当前域检查 EDIT 权限，之后才合并请求；调用者不能只在请求里填一个自己有权限的域，就替代对原对象的授权判断。这证明的是原对象修改授权的取值顺序，不等于已证明目标域迁移受控。

第一层追问：能否据此说用户不能把对象迁到任意域？答：不能。当前 _check_protected_fields() 明确保护 name 以及数据集的 dataset_name/version，creator 在后续单独保护；这不等于 Domain 也不可改。在本段尚未看到合并后目标域的二次授权，需继续核对 API、SDK guard 和下游写入规则，再判断迁移约束是否完整；也不能仅凭这里没有检查就宣布已证明存在漏洞。

第二层追问：怎样验证，而不是停在推测？答：【改进建议】构造虚构域 A/B：用户可改原域 A 但不可写目标域 B，分别走 API 与直接 SDK，检查拒绝位置、实际写入与审计；另覆盖没有原域权限却伪造请求域的用例。原对象授权、目标域迁移授权和检查至写入的 TOCTOU 是三个问题，需分别验证；版本条件或事务是并发改进方向，不是当前已排除竞争的证据。

Q7【进阶】为什么 Java、Play 和 React 都要改？

答：GMS 承担身份和权限，Play 承担服务端会话与代理凭证，React 承担用户操作；只改其中一层会产生登录成功但 API 身份不一致等断点。

第一层追问：认证与授权的区别？答：认证证明谁，授权决定其对资源的动作。

第二层追问：白名单与角色重复吗？答：准入是粗门槛，角色与域是细操作权，不能互相代替。

Q8【深入】长效 PAT 自动供给如何避免重复和泄露？

答：使用稳定名称/实时 secret 读取、每用户锁与二次检查；服务端用调用者权限读写加密 Aspect，代理使用时注入。

第一层追问：加密与 hash 为什么都可能需要？答：验证可只用 hash，再次取回原值则需受控可逆存储，两者用途不同。

第二层追问：撤销后缓存呢？答：缓存可能短期陈旧，应结合失效、短 TTL 或下游拒绝处理；这是需要核验和改进的边界。

Q9【深入】Agent 已登录，为何还说读权限待验证？

答：登录只保护入口；读取范围取决于工具以谁的 token 访问什么资源。注册有 can_register 门禁，但不能推导所有读工具都逐用户隔离。

第一层追问：注册被拒后模型重试参数怎么办？答：权限错误不是格式错误，工具继续拒绝并向用户解释。

第二层追问：要证明没有越权，需要什么？答：不同用户、不同域的正反例，检查真实下游身份和审计结果，覆盖直接 HTTP 与工具路径。

Q10【深入】如何证明重构有价值，而不是代码变多？

答：展示一条实体规则如何被 HTTP 和库调用复用，列出 SDK 消费方、独立测试和错误隔离；用具体维护与接入问题说明价值。

第一层追问：代码量能代表贡献吗？答：只能辅助盘点，模块职责、设计决策和可验证补丁更有说服力。

第二层追问：你最愿意承认的不足？答：跨副本一致性、权限边界和异步可见性仍需系统验证；明确不足比把基础组件说成原创更可信。

8. 两个白板练习

练习一：两个进程注册同一 URN。 画“查不存在→发 A→发 B→验证”，让一进程在 A 后超时，另一进程覆盖。答案须指出：进程锁不跨进程，重试不回滚，存在不证明 B 正确；再提出版本约束或幂等键并说明代价。

练习二：登录后调用 Agent 注册。 逐跳标 actor、session token、PAT、can_register、目标域与 creator。篡改 actor 头、撤销角色或传别人的 session_id 时，哪层应拒绝？按身份表区分已有检查与待测边界，页面按钮不作授权证据。

9. 源码阅读路线与个人证据

本地源码根：`D:\博士后出站考核\相关项目\data_infra_platform`。先读 `docs/architecture/数据工程架构设计说明书.md` 看团队功能域；再进入 `libs/datahub_extensions/`：

1. `datahub_register/metadata/entity/metadata_entity.py`：URN、校验、MCP 构造，确认纯计算边界。
2. `datahub_register/metadata/service/metadata_service.py`：register、amend、锁、exists 与 verification。
3. `datahub_register/metadata/data_access/`、`publish/`：GraphQL、搜索和 Emitter 分工。
4. `datahub_extensions_api/api/routers/register.py`、`permission/dependencies.py`：HTTP 主体与域权限。
5. `datahub_extensions_api/api/routers/chat.py` → `datahub_chat/chat/service/chat_service.py` → `mcp/tool_adapter.py` → `tools/register/register_metadata.py`：Agent 全链路。
6. `third_party/datahub/metadata-service/auth-servlet-impl/src/main/java/com/datahub/auth/authentication/IDaaSAuthController.java`：企业身份入口；文件名是本地历史命名。
7. `third_party/datahub/datahub-frontend/app/auth/nevertoken/NeverTokenManager.java` 与 `app/controllers/Application.java`：凭证和代理。
8. `third_party/datahub/datahub-graphql-core/src/main/java/com/linkedin/datahub/graphql/resolvers/auth/` 与 `datahub-web-react/src/app/`：服务端凭证读写和浏览器页面。

【已实现】个人补丁入口：`fe241aca` SDK/API；`b6f7ef59` 身份；`e2013a93` 准入；`360cc996` 角色；`9dd6c201` 凭证；`65b1edb6` 对话；`523ed8e2` 配置和链接。用 `git show <hash>` 读具体改动，区分后续团队补丁。

03 推理网关：路由决策、Redis 状态与 callback 正确性

私人学习材料，2026-11-02。对应面试稿第 15—17 页和路由备份页。源代码以本地 omni-elb/omni-elb 快照 25f88a3、个人补丁和合成复现模型为依据。内部服务地址、凭证、员工编号和真实业务规模不进入本章。

标签：【已实现】有代码依据；【理论/复现】分析模型或独立模拟；【通用知识】系统原理；【改进建议】可讨论的后续设计，不冒充现有能力。

1. 大项目、LiteLLM 与个人职责

【已实现】Omni-ELB 项目存在不同实现方向，包括 OpenResty/Lua 数据面与控制实现，以及基于 LiteLLM 的网关方向。面试讲我参与的 LiteLLM 路径，不把它们硬接成一条流水线。

这一方向服务模型应用开发者和基础设施团队：客户端通过入口代理访问统一模型 API，LiteLLM 承担调用凭证、模型组、健康候选、策略与转发，连接多套推理后端。Redis 提供共享状态，PostgreSQL 支持网关持久数据，Prometheus 提供观测；状态刷新与弹性服务属于旁路控制链路。入口代理负责转发和连接管理，不能把全部模型凭证能力归给代理。

我的职责分两层：会话亲和和负载感知路由由我与同事共同设计；独立 Redis 刷新和 PipelineKey 重复计量修复有本人直接提交。不能把团队网关归为个人独立开发，也不能因只找到两项合入就把协作算法职责缩成两次修 bug。

一分钟讲述：

> 我参与基于 LiteLLM 的多后端推理网关，重点是选点和其依赖的状态。我们结合会话复用与低负载候选，使新会话分散到后端，同时保留兼容路由。我直接实现状态自动发现与刷新，修复指标被重复记录的问题。关键经验是：路由算法好坏取决于读到了什么状态，以及真实 callback 调用路径是否符合预期。

【通用知识】数据面处理每条请求；控制面维护配置、健康与资源状态；观测面记录行为。它们相关但不必同步串行：把 Prometheus 查询放入每请求路径会引入额外时延和依赖，使用 Redis 快照则要承担陈旧状态风险。

2. 按函数体记住真实路由树

【已实现】核心入口是 router/api_key_router.py::async_filter_deployments()，在健康候选之后、实际转发之前执行。顺序是：

1. 先提取 user_id。优先 litellm_metadata.requester_metadata.user_id，再 metadata.user_id，最后 litellm_session_id。
2. 有有效键，走会话/负载分支。Redis 中已有映射且目标仍在健康候选中，就直接返回原部署。
3. 需要重选时才 MGET 各部署在途键。有可用 counts，从最低负载候选中随机选；候选数是 max(1, ceil(0.10*N))。
4. counts 不可用时，使用确定性 hash 兜底，并把新映射写回 Redis，带抖动 TTL。

5. 无 user_id 时检查 PipelineKey；命中则返回原健康列表，让框架后续策略继续选择。它不是插件内部固定选择列表第一项。
6. 其余请求按 API key hash 等路由键做确定性选点；单候选和异常有对应回退。

`_is_pipeline_key()` 当前函数体也把 alias 缺失视作该路径，因此字段缺失可能改变分支，不能只按“业务上叫 PipelineKey”理解。传入 user_id 的优先级高于 PipelineKey。某些 docstring 仍写“多次加盐避开最忙部分”，但当前 `_route_user_by_least_busy()` 实际调用 `_pick_least_busy()`；后者注释的 30% 也旧了，函数体是 10%。

【通用知识】最低候选随机有助于避免多个网关同时总挑同一台，不过读快照再选点仍可能并发扎堆，不是强原子负载均衡。确定性 hash 提供相同输入与候选集下的可重复性；不自动等同具有少量迁移性质的一致性哈希环。

【改进建议】对于缺失或坏 counts，应区分“未知”与“确认空闲”。当前部分回退把缺键按默认零参与排序，可能吸引流量；加入状态年龄和可用性标记后才能更细地治理这种风险。

3. 状态刷新为什么比手写模型列表可靠

【已实现】个人独立刷新实现位于 `litellm/k8s/refresh-redis/gen_inflight_script_standalone.py`。旧方式维护模型名正则，名字改变可能静默漏刷。改进后的主路径从 `/model/info` 获取模型组与 `model_info.id`，查询监控中的 P 侧在途，再用稳定 ID 关联，生成回写 Redis 的指令。

关联契约是：网关部署 ID 与监控 cluster 标签对应。主路径按服务角色拉完整负载，再按实际集群 ID 筛选，避免依赖模型展示名；模型别名与 upstream 名只是辅助字段。代码也有明确的名称归一化回退与配置降级，不应宣传成所有部署永远严格同名。

【已实现】P 侧负载取 `running + waiting`。先在 `(cluster, instance)` 内聚合多条 engine 序列，再对 running 和 waiting 做 outer join、缺边填零，最后按 cluster 求和。原因是某实例可能只有一类指标；inner join 会丢掉单边存在的真实负载。如果两边都没有，不等于负载为零，更不能直接断言后端未运行。

例：实例 A 的 running=3、waiting 缺失；实例 B 的 running 缺失、waiting=2。正确代理负载为 5，inner join 可能把两者全丢。这个例子是合成演示，不是真实业务数据。

脚本对未匹配项列出诊断，有些历史日志把某类缺失命名为“未运行”，面试应解释它只是候选原因，还可能是未采集、标签变化或观测窗口缺失。配置发现失败时可以退回显式配置，但日志需说明实际来源，不能以降级后有输出证明自动发现成功。

【通用知识】请求数只是工作量代理。两个请求的 prompt 长度、缓存命中和生成长度可能差很多；P 的 running/waiting 也无法完整代表 D 瓶颈。请求路由选集群，集群内部 P/D 资源调度另有职责，不把 D 扩容收益归给入口选点。

4. Redis 物理键、花括号与两类 TTL

【已实现】在途逻辑键为 `{model_group}:deployment_id:inflight`；路由映射键为 `user_route:user_id`。这里花括号是物理字符串中的字符，同时是 Redis Cluster hash tag：同一模型组的

候选键可以落到同一 slot，便于相关多键操作。实际物理键还可能经过 `check_and_fix_namespace()` 加前缀，必须逐字比较读写两端，不能只比较打印的模型名。

别名在不同阶段解析，会造成路由读取规范模型组的键，而增减写到原始别名键；两个键都存在但彼此隔离。稳定 ID、规范模型组和 namespace 要一起核对。全局 sticky 键是否包含模型组，也是设计契约问题；不能在面试中未经代码核查就画成按模型自动隔离。

【已实现】路由映射 TTL 决定绑定持续多久，带抖动减少集中失效；它不是 KV cache 的真实驻留时间。在途键 TTL 是丢失释放时的回收保险，当前在计数增加时刷新 TTL。持续有新请求时 TTL 可一直被续期，故它不能保证漂移误差迅速归零。

【通用知识】Redis `INCR` 原子不等于“读多个负载→挑最小→预留”整体原子。`least_busy_fix` 的某些入口用 Lua 完成选择加预留；APIKeyRouter 的 MGET 低负载候选分支不是同一段原子脚本。是否走该入口依赖真实配置和 hook，不应把模块存在说成所有请求都用了 Lua。

当前源码中 `_is_allowed()` 直接返回 True，附近“空 allowlist 则休眠”的旧注释不能当作当前开关逻辑。面试先说快照事实，再说明部署状态还需看配置，避免对文档、代码和真实启用状态三者混说。

5. 计数漂移、取消与周期校准

【已实现】计数模块在选择/预留时增加，成功、异常或流式生命周期中释放。当前代码已有首 chunk/P-only 条件释放、流式 finally 和取消屏蔽；不能把历史 `CancelledError` 泄漏动机写成当前完全没有处理。若配置只计 P，首 token 后释放与计到整个 E2E 结束的含义不同，必须说清负载边界。

【通用知识】理想守恒是 **初值 + 已预留 − 已释放 = 当前在途**。进程被强杀、补偿访问 Redis 失败、某分支遗漏、重复回调、重试/别名写错键，都可能破坏它。取消不是普通业务失败的同义词；异步清理可能在 await 处再次被取消。finally、shield 和幂等释放可减少风险，但无法让进程强杀执行清理。

【已实现】周期监控快照 SET 覆盖提供校准，不是每请求计数的替代品。它能把累积误差拉回观测附近，却引入采集、查询、生成、执行等时间差。SET 与并发增减交错也可能覆盖刚发生的变化。

【改进建议】可把状态拆为带时间戳的观测基线和后续增量，处理版本、原子更新及归零边界；也可以增加状态年龄门槛或限流兜底。直接把刷新频率提高若干倍不是完整答案，还要考虑采集分辨率、控制开销和读写竞争。生成脚本中的时间是快照时间，不是执行时的实时状态。

6. 指标翻倍的真正根因

【已实现】`pipeline_key_log_filter.py` 过滤部分请求的 DB/归档日志。旧补丁在跳过 DB 日志时，手动找到 PrometheusLogger 并调用其 success 方法，意图“日志不落库，但指标保留”。然而上游成功处理链本来就会把 PrometheusLogger 作为独立 callback 调一次，于是同一请求被计两次。

个人提交 `d387aa9` 删除手动触发 Prometheus 的逻辑；过滤器只负责日志过滤，计量继续由独立 callback 完成。它修的是职责重复，不是把监控结果除以二。否则正常请求、异常分支和其他 callback 仍可能不一致。

【通用知识】counter 是调用次数的累积，不会理解业务上的“同一请求”。排查应画 callback 调用图、比较一条请求前后的样本增量、检查标签和成功/失败/stream 分支。进程多实例和 Prometheus 抓取重复也会造成表观翻倍，但不能用通用猜测取代本次已确认的函数调用重复。

【已实现】logger patch 已有单次安装保护。【改进建议】核验现有保护，并补充多次实例化/导入的回归，使用 mock callback 断言每个事件的调用数，再以隔离端到端请求检查 counter 增量。本章未重新运行这些网关测试，不能表述为本轮已完成线上验证。

7. 算法一与算法二：生产协作与理论必须分开

【已实现】算法一对应会话保持、新会话低负载候选和兼容分支的工程主线，属于协作算法。代码运行状态和可披露效果需分别核对；不能把生产改善归为某一个修复，更不能与专家均衡论文消融收益相加。

【理论/复现】算法二针对已有会话：比较迁移到候选 B 的等待收益和失去原后端 A 缓存的代价。合成复现中的简化判据为：

`W_A - W_B > (1+alpha) × hit_A × mean_prompt / prefill_rate`。

把它从预计完成时间推一遍，比背公式有用。令原绑定 A 和候选 B 的预计等待分别为 W_A, W_B（秒），原后端近期 token 命中比例为 h_A，平均输入长度为 P（token），共同的预填充速度为 v（token/秒）。候选 B 的可复用缓存未知，决策近似按零处理；先假设其他阶段耗时相同并抵消，则

$$\hat{T}_A = W_A + (1 - h_A)P/v, \qquad \hat{T}_B = W_B + P/v.$$

要求 $\hat{T}_B < \hat{T}_A$，移项得到 $W_A - W_B > h_A P/v$：切换节省的等待必须补回失去 A 缓存的预填充成本。再加入非负相对裕度 α，就得到上面的判据。源码变量 `hit_A`、`mean_prompt`、`prefill_rate` 分别对应 h_A, P, v；这里的命中率是后端历史统计近似，不保证本请求恰好命中相同比例。

合成手算例子，非业务数据：设 $P = 1000$、$v = 2000$、$h_A = 0.8$、$\alpha = 0.2$，估计损失的缓存价值为 0.4 秒，加裕度后门槛为 0.48 秒。若 $W_A - W_B = 0.6$ 秒，判据建议切换；若只有 0.3 秒，则保持。恰好等于门槛也不满足严格大于。若后端速度不同、B 实际已有缓存或 D 阶段代价不同，这个简式就不足以完整比较。

左边与右边的单位均为秒。B 取最小预计等待候选；缺少缓存统计时保持原绑定。它用后端统计而不读取未观测的 B 真实缓存，也不偷看未来输出长度。执行模拟再计算真实模型缓存命中，因此可以暴露这个近似的得失。

【通用知识】切换不只影响 P：还可能有 KV 传输、D 等待、不同算力和重试代价；状态陈旧时等待收益可能被高估。【理论/复现】更一般的双端缓存、异构速度、误差裕度分析不自动成为当前代码保证。`sticky_aware` 的在途阈值基线又是另一项实验候选，不等同于完整算法二。

Simulator 作为线上链路之外的实验工具挂载真实插件；`llm-scheduling-repro` 则自行实现合成模型。二者不能混成同一个公开代码版本或共用性能结论。

8. 十组深挖问答

Q1【基础】网关为什么还需要自定义路由？

答：统一 API 不自动解决特定会话语义和状态接入。我们在健康候选后加入会话与负载决策，保持框架的转发与兼容能力。

第一层追问：你改了 LiteLLM 内核吗？答：重点通过 CustomLogger hook 和回调适配，部分 patch 要逐处说明，不能笼统说完全零侵入。

第二层追问：其他 Omni-ELB 实现在哪？答：属于并行方向，本介绍只覆盖自己参与的 LiteLLM 路径，不虚构统一部署链。

Q2【基础】user_id 与 PipelineKey 谁优先？

答：有效 user_id 优先；无键且是 PipelineKey 才旁路返回健康列表。user_id 的提取也有多级位置。

第一层追问：alias 缺失呢？答：当前实现也判为 pipeline 路径，字段缺失可能改变行为。

第二层追问：返回多个候选算没路由吗？答：是插件让后续框架策略继续选，不是整套网关没有路由；仿真若总取首项会错误。

Q3【进阶】为什么最低 10% 随机，不选严格最小？

答：用候选随机缓解同快照下的竞争与抖动，同时让候选仍偏闲；10% 是实现参数，不是性能改善。

第一层追问：N 很小时？答：向上取整并至少一台，N=9 时只有一个候选，随机不会产生多选效果。

第二层追问：能证明最优吗？答：不能，负载代理、陈旧状态和缓存都影响实际耗时，需受控对照与退化场景。

Q4【进阶】为什么稳定 ID 比名字可靠？

答：名字有展示名、别名、upstream 名和监控名多层语义；ID 直接对应部署，但仍需核验契约与少量归一化例外。

第一层追问：监控没找到等于没运行？答：不等于，可能缺采集或标签变化。

第二层追问：outer join 为什么必要？答：running/waiting 单边存在时 inner join 丢负载；缺指标与确认零值还要分开。

Q5【进阶】花括号为什么不能删？

答：它是物理键的一部分，也是 Redis Cluster hash tag。删除会改变键名和 slot，读写不匹配，多键脚本还可能跨 slot 失败。

第一层追问：namespace 加前缀会破坏 tag 吗？答：通常正确保留花括号则不破坏，但要比较真实拼接结果。

第二层追问：别名写错怎么办？答：同一 ID 也救不了组名错键；沿读者和写者分别追踪规范化阶段，不能把所有键求和蒙混过去。

Q6【进阶】TTL 能解决计数漂移吗？

答：能限制闲置错误键的持续时间，活跃键持续续期仍可能保留漂移，所以还需要校准与生命周期修复。

第一层追问：finally 不就可靠了？答：进程强杀不会运行，清理中的网络失败或再次取消也要处理。

第二层追问：当前取消处理做了什么？答：当前有流式 finally/shield 和补偿路径；仍要验证每种调用路径恰好释放一次，不能只看存在 finally。

Q7【深入】为什么周期 SET 会和 INCR 冲突？

答：快照反映过去时点，SET 会覆盖期间的增减；它是校准近似，不是线性一致的当前真相。

第一层追问：用 Lua 包 SET 就好了吗？答：只能保证写入步骤原子，不能消除快照年龄。

第二层追问：更合理的设计？答：观测版本加增量、过期状态隔离或误差裕度，明确复杂度和故障恢复，不能当作现有实现。

Q8【深入】Prometheus 指标为什么翻倍？

答：过滤 DB 日志的补丁手动调用 Prometheus，而框架独立 callback 已调用一次。删除重复调用，恢复职责分离。

第一层追问：为何不在图表除二？答：那掩盖根因，别的分支可能只计一次。

第二层追问：怎么验证？答：隔离一条请求，记录各 callback 触发数和指定标签 counter 增量，再覆盖普通、pipeline、stream 与异常路径。

Q9【深入】算法二缓存阈值怎么推出来？

答：把留在 A 和转到 B 的预计 TTFT 相减；在目标缓存未知的简化下，迁移收益需要超过 A 的可复用预填充时间及裕度。

第一层追问：把 hit 当请求命中概率吗？答：复现中是近期命中 token/输入 token 的统计，应保留定义，不能随意替换。

第二层追问：B 也有缓存或 D 堵了？答：简化判据可能保守或失真，需双端缓存、异构及全链路等待模型；当前并非线上已部署能力。

Q10【深入】怎样证明这是你做的？

答：协作算法按已确认职责描述，直接实现用刷新和双计数补丁解释问题、选择、代码与影响；不给没有实验条件的数字。

第一层追问：提交少是否贡献小？答：提交数不反映协作设计及故障难度，展示模块与关键决策。

第二层追问：团队结果如何归因？答：系统结果归团队，个人说负责的模块、证据与可验证作用，避免把全部吞吐或时延变化写成个人结果。

9. 两个手算与排查练习

练习一：路由与键。 设 12 个健康候选，最闲两台在途为 0、1，其余更高；新会话候选数为 2。画出 sticky 命中、sticky 失效、Redis 不可用、PipelineKey 无 user_id 四条路径。把规范模型组 demo、部署 c1 的物理逻辑键写成 `{demo}:c1:inflight`。再给读者组名 demo、写者组名 demo_alias，解释为何计数看似有值但路由看不到。假设数据不代表真实部署。

练习二：取消与计量。 一条流式请求在首 chunk 前断开，另一条在首 chunk 后断开。分别按 P-only 和计完整服务两种配置，标出 +1/-1/日志/Prometheus callback。指出重复释放如何变负数、未释放如何漂移；最后加入周期 SET 的时间线。合格答案必须区分当前已处理路径与仍需测试的交错。

10. 源码路线、证据与口头禁区

本地根：D:\0608lc\omni-elb\omni-elb。阅读顺序：

1. litellm/router/api_key_router.py：hook→键提取→sticky→MGET→候选→hash，逐项对照函数体。
2. litellm/k8s/refresh-redis/gen_inflight_script_standalone.py：发现、prom_load_all、outer join、lookup_load、生成脚本。
3. litellm/router/least_busy_fix.py：键、Lua reserve、增减、stream/finally、patch 安装；核验是否进入当前配置路径。
4. litellm/router/pipeline_key_log_filter.py 与提交 d387aa9：直接读删除的手动 Prometheus 调用。
5. litellm/config/ 与 docker/：只查 callback 和调用链，不把配置/日志中的秘密复制到备战材料。
6. D:\博士后出站考核\llm-scheduling-repro\src\llm_repro\routing.py::choose_backend 和 docs/MODEL.md：算法二公式、缓存定义与合成模型边界。

【已实现】个人证据：2967aea 是独立状态刷新脚本，d387aa9 是 PipelineKey 重复计数修复。协作算法范围以主计划 §5.3、§15、§16.3 的本人确认及指定架构为准；不能把整个 least_busy_fix 或全部最新源码改动都列为自己独立实现。

不要说：hash 一定均匀、sticky 一定命中真实缓存、P 请求数等于剩余工作量、TTL 保证无漂移、Lua 存在就表示所有路径原子、生成快照等于实时负载、算法二已经上线、合成模拟证明线上收益。正确说法要带上状态、路径、指标与适用条件。

04 路由 Simulator 的建模与工程验证

本章对应网关项目中的请求级服务仿真器。依据源快照 25f88a3 与逐函数核查整理。它直接加载真实路由插件，维护模拟请求与在途状态，再由可替换后端产生请求级 E2E 和 TTFT。其源码不在 llm-scheduling-repro 包内；后者是另一套显式 P/KV/D 合成模型。

1 核心问题与个人工作

【已实现】我承担的是实验基础设施：采集契约、插件接入、事件引擎、状态适配、参数化/经验后端、校准和运行编排。团队已有网关和真实路由插件；模拟器复用这些策略，而不是据此声称重写整个网关。

为什么需要它？线上策略比较常同时改变请求、后端状态和模型版本，单看上线前后均值不容易解释原因。另写一份路由算法又可能与实际插件分叉。模拟器希望固定可控部分，检查“插件是否真正参与”“状态是否重建合理”“新落点的耗时靠什么假设得到”。它适合接口回归与机制研究；要证明真实容量或策略普遍优越，还需要独立设备和流量验证。

面试一分钟说明可用：“我把真实插件与事件引擎、状态存储和耗时模型拆开。引擎只知道请求到达、派发和完成；路由仍由原插件选择，Backend 返回服务耗时。这样可以分别检查代码一致性、状态口径和反事实估计。最重要的边界是，给定真实落点并复用真实时延的回放，不能拿来证明预测能力。”

2 数据模型与执行主线

2.1 三个接口边界

【已实现】TraceRecord 规范化请求 ID、到达、输入/输出 token、E2E、模型组等字段；历史落点、TTFT、用户/会话键是可选信息。缺字段会缩小可验证范围：没有历史落点就不能计算落点一致率；只有聚合监控，无法重建完整会话轨迹。

插件桥接层按原模块路径加载 handler，给它准备请求形态、健康候选和隔离状态。仅导入成功还不够；bootstrap 用带会话的探针和候选列表检查插件是否真正返回单部署，防止旁路后引擎总拿第一个候选。这个探针没有构造不对称负载，不能说它已经验证所有负载分支。

Backend.serve 接收输入/输出规模、调用时在途数、缓存近似与随机源，返回 `duration_ms` 和 `ttft_ms`。接口明确 inflight 不含本请求。经验后端还可以接收真实落点和耗时，用来判断是否能复用观测；这恰好形成条件回放与未知落点估计的分界。

2.2 完成事件驱动状态变化

【已实现】派发前先释放已经完成的事件，更新相关集群在途，构造插件输入并同步状态；插件选中目标后调用 Backend，将完成时刻压入最小堆。选中后在途加一，完成时减一。事件时间用秒，后端耗时用毫秒，入堆前必须换算。

可以用下式记住最小模型：

$$e_i = t_i + d_i/1000, \qquad n_c(t) = \sum_i 1\,[c_i = c,\ t_i \le t < e_i].$$

这里没有显式 P 队列、KV 链路、D 批调度和逐 token 内核。拥塞效应由后端公式或经验分布近似。事件堆保存完成时刻，并不自动产生真实引擎队列；如果服务模型不随并发变慢，单纯加并发就可能得到不真实的线性吞吐。

【通用知识】最小堆支持取最早完成事件。处理 M 条请求的事件维护通常是 O(M log M)，但插件排序、Redis I/O、经验池拟合和快照复制另计。相同时刻的比较和“先释放后到达”的顺序要固定，不能每次运行依赖不稳定容器顺序。

3 开环与闭环怎样回答不同问题

【已实现】开环按输入轨迹给定到达，不等待上一请求完成。到达缩放控制输入压力，服务缩放改变模型执行的时间尺度。同步压缩两者只是模拟时钟变化；只压缩到达则增加相对压力。路由中的墙钟 TTL 不一定随模拟时钟同步变化。

【已实现】闭环容量模式先填到并发水位 K，完成一条后补一条。它保留请求组成与顺序，但不保留原开环间隔。该分支不向经验后端传实际落点，避免一部分策略走真实值复用而另一部分只能估计。若插件返回空部署，当前容量分支没有完整记录拒绝与补位语义，因此不能用它可靠比较拒绝策略。

【通用知识】稳定闭环系统可以用 Little 定律解释平均在途、完成率和平均停留时间的关系，但有限轨迹存在启动和排空。**完成数/首请求至全部完成** 是有限运行完成率，不自动等于稳态极限容量。若后端只按在途调整耗时而没有资源瓶颈，水位扫描的“饱和点”首先是模型现象。

4 经验后端真正知道什么

4.1 观测复用不是预测

【已实现】当选中集群与历史落点相同、记录耗时有效时，经验后端直接复用观测。`exact_replay` 还会在每次派发前把该请求的真实落点写入会话映射。读该映射的策略因而接近历史落点，后端随后复用真实时间。

这能检查字段、调用链和结果管线，却含有答案标签。不能把落点一致率称为独立路由预测准确率，也不能用时延分布接近证明未知策略的反事实时间准确。一个“已知答案输入→相同答案输出”的流程不等于对未见样本有解释力。

4.2 未知落点依赖条件经验池

【已实现】未知落点从按 prompt 长度/在途条件组织的样本池中取样。样本不足或新集群可借参考池，再按当前和样本时在途的期望耗时比例校正，并做上下界截断；必要时回落到参数化后端。TTFT 与 E2E 有独立校正。

这些假设要能逐条说出来：相同分桶内样本具有一定可迁移性；在途数能解释部分变化；线性趋势及截断足以描述覆盖区间；参考集群的基础能力可借用。输出长度没有被独立作为经验采样条件，不能因为接口接收 completion_tokens 就说完整建模了不同解码长度。

拟合曲线使用在途桶均值。桶均值上很高的 R^2 不意味着逐请求预测误差小；桶内方差可能很大，训练与检验还可能共用数据。不同策略进入不同负载和缓存区域，误差可能对一个策略乐观、另一个悲观，所以共享同一 Backend 也不能保证策略排序正确。

5 状态与缓存近似的边界

【已实现】引擎在派发时把其掌握的在途状态同步给插件，通常比周期采样的线上状态更及时。不能默认离线结论已经包含线上刷新延迟。批量读写、变化集群同步和内存模式减少状态访问开销，但目前没有据此新增可公开的加速倍数。

进程内 Redis 替身没有执行完整 TTL 过期，Lua eval 未实现。它适合接口覆盖足够的受控扫描，不是多客户端原子性和键过期的等价实现。真实 Redis 使用墙钟时，即使接口是真实的，时间语义仍可能与快速推进的模拟时钟不同。

缓存又有两层：路由 TTL 只控制“会话还记不记得原目标”，设备 KV 驻留还受容量、淘汰和前缀内容影响。本模拟用会话记忆和折扣近似后者，没有逐块管理真实 KV。指标中 `apc_hit_rate = sticky_hit_rate`，应表述为模型内代理，而不是设备 APC 实测。

6 指标和归档必须讲到分母与单位

【已实现】TPS 以选中请求数除以排空跨度，再按时间尺度换算；单位是请求/秒，不是 token/秒。开环长窗口里这个值可能接近外生到达率，不适合独自评价容量。不同模式的拒绝漏记也会影响分母。

TTFT 分位函数先排序，取零基下标 `min(int(n*p/100), n-1)`，无插值。n=20、样本 1 到 20 时 P95 返回 20；nearest-rank 取第 ceil(0.95n)项会返回 19。这不是哪一个名字更“标准”的争论，而是跨代码比较时必须统一定义。本仓库 llm_repro 采用的口径与此处不能默认相同。

瞬时在途 CV 先对每个快照的集群负载求标准差/均值，再等权平均非零均值快照；它不是按持续时间加权。累计分配 CV 只看各集群最终分到多少请求，不能表达某一分钟的热点。当前回放排空阶段快照不完整，精细负载分析应从派发与完成事件重建。

预填充节省是冷态模型 TTFT 与执行模型 TTFT 的差分，并受取样噪声影响；不是实测“省了多少时间”。落点一致率的分母仅含有实际落点且已选中目标的请求，需要同时说明标签注入和预热。

默认 CLI 归档保存代码提交、种子列表、运行元数据和汇总指标。桥接对象的插件指纹及 doctor 打印不代表已写进默认 meta；逐种子请求明细、完整拟合参数、输入哈希也不能假定全部保存。学习实验应补上这些信息，再谈严格可复现。

7 分级问题与继续追问

S01 为什么直接挂载插件而不重写算法

答：重写容易让离线版本和真实服务分叉。直接挂载保留原 handler，但还需复现输入、状态和调用次序。我的工程工作是把桥接与引擎、后端分开，并用会话探针检查是否选出单一目标。

追问一：同一源码是否就证明等价？不。环境、状态接口、健康候选、key 提取和 hook 时序不同仍可能改变结果。应做固定输入状态的分支与落点对照。

追问二：探针通过是否能替代完整回归？不能。当前探针主要防旁路；还要覆盖无会话、无状态、异常、TTL 及随机分支。

S02 为什么用事件堆而不是每毫秒扫描

答：关注的是请求派发和完成时状态变化，用堆直接跳到下一事件避免空扫描。时间分辨率不由固定 tick 限制，事件维护规模与请求数相关。

追问一：它有没有真实队列？当前维护完成事件和在途状态，服务时间由 Backend 给出；并不等同引擎的资源队列。

追问二：同时到达和完成怎么办？先固定释放规则与稳定 tie-break，并用微场景验证计数与落点。不能让容器偶然顺序决定结果。

S03 开环 TPS 高是否说明算法容量高

答：不能直接这样解释。开环输入决定请求到达，有限轨迹完成率还包含排空。需要闭环水位扫描或受控饱和负载，同时报告尾时延、拒绝与模型条件。

追问一：闭环是不是生产最大容量？也不是，它只对应 K 及服务模型。需要验证模型对拥塞和资源竞争的描述。

追问二：K 翻倍、TPS 也翻倍是否好？先检查 Backend 是否缺少并发退化或容量约束。可能只是模型允许无限并行。

S04 高落点一致率为什么仍不能证明预测能力

答：若逐请求真实落点已写回会话映射，预测目标的一部分已作为输入；相同落点又能触发实测时间复用。应先说明它是条件回放。

追问一：如何独立验证？关闭标签注入，以训练窗口构建模型，在时间留出的未见流量上检验。

追问二：只留出时间就充分吗？还需检查新策略到达的负载、缓存和集群区域是否被训练覆盖，必要时受控实机对照。

S05 共用同一个 Backend 为什么不保证排名

答：策略改变落点会改变在途和会话局部性。误差是条件相关的，不一定是给所有策略加同一个常数。共同后端减少条件差异，但无法消除分布移动。

追问一：怎样发现排序不稳？扫缓存折扣、负载校正、弱样本参考池等合理参数范围，报告排名反转和误差带。

追问二：最终选哪个？先排除依赖外推的假收益，再做小规模独立验证；如果证据不足，应保留候选而不是宣布赢家。

S06 R^2很高为什么不能说逐请求很准

答：当前拟合的是分桶均值，平均化会掩盖桶内离散。R^2 也不直接衡量尾分位误差和留出误差。

追问一：补什么指标？按负载和 prompt 分层的绝对/相对误差、P50/P95 误差、回落比例及覆盖范围。

追问二：同数据拟合和检验会怎样？会高估泛化，应区分训练、时间留出及新策略验证集。

S07 会话命中为什么不能直接当 KV 命中

答：会话同目标只说明复用机会，KV 可能已淘汰，前缀也可能变化；不同会话还可能共享公共前缀。本模块用会话记忆作近似，必须声明。

追问一：怎么改？引入有限前缀块缓存、驻留/淘汰和内容对应关系，并用引擎计量校准。

追问二：本仓库是否已经做了这件事？llm_repro 有独立有限 LRU 前缀模型，但那是另一实现，不能倒写成本 simulator 已有。

S08 内存 Redis 更快，为什么不能直接替换所有实验

答：它减少往返，但没覆盖 TTL、Lua 和多客户端原子语义。优化性能不能靠悄悄删掉会影响策略的语义。

追问一：真 Redis 就一定对吗？墙钟 TTL 仍可能与模拟时间不同，实验时间尺度要统一。

追问二：如何做可信替身？先列接口和时钟契约，再对同一状态序列比行为，未覆盖功能显式拒绝或标记。

S09 固定一个 seed 是否就是公平对照

答：不同策略可能消耗随机数的次数和顺序不同，单个 seed 不能保证每请求配对同一扰动。应固定轨迹并重复，最好按请求及噪声类型分离随机流。

追问一：随机流怎么设计？可用 run seed、request ID、噪声类别派生，避免策略分支改变无关随机量；这是改进建议，非当前默认保证。

追问二：多 seed 均值还缺什么？方差、尾部、退化场景及配对差值；样本少不能据此宣称普遍显著。

S10 当前指标最容易漏掉什么

答：容量模式空部署返回未完整记拒绝，快照 CV 非时间加权，分位规则不同，APC 为会话代理。必须能指出这些口径对结论的影响。

追问一：如何补拒绝？先定义拒绝是否消耗闭环补位/时间，再保存逐请求原因，保证输入、选中、拒绝与完成对账。

追问二：为什么不能只修输出表？分母与事件推进本身可能错，后处理无法恢复未记录的事件。

S11 如何证明我的模拟器实现正确

答：分层验证。先测固定时长事件和守恒，再验证插件调用与状态接口，再验证后端在独立数据上的误差，最后验证策略结论。校准 4/4 自检只支撑构造场景的分类行为。

追问一：接口过了为什么时间还可能错？单位、到达/服务缩放、真实时延已含排队而又乘拥塞等都可能重复计时。

追问二：线上效果和模型差很多怎么办？先检查输入与状态口径，再检查后端覆盖，最后考虑策略本身；不要直接调参数直到表面贴合。

S12 面试问你真正交付了什么，如何回答

答：我实现并迭代了真实插件接入、采集/校准、事件与后端解耦、时间尺度、亲和基线及状态访问优化。可指向对应提交与函数。效果以已运行的接口/合成检查描述，不编硬件收益。

追问一：最有价值的失败经验？条件回放可以看起来很准，却不回答反事实问题；必须先确定实验究竟要验证什么。

追问二：下一步优先什么？补拒绝与逐请求归档，关闭标签注入做时间留出，再用有限范围实机对照验证模型和排序。

8 两个可自行完成的练习

练习一 固定时长的闭环事件与计数

【理论/复现】下面是独立教学模型，不调用原插件、不访问 Redis。8 条请求，闭环水位 2，每条服务 2 秒，预期最后完成 8 秒，完成率 1 请求/秒。它没有资源瓶颈，所以增大水位会继续增加模型完成率；请解释这为什么不是实机扩容结论。

```
import heapq

def closed(n, k, service):
    q, started, finished, t = [], 0, 0, 0.0
    while started < min(k, n):
        heapq.heappush(q, (service, started))
        started += 1
    while q:
        t, rid = heapq.heappop(q)
        finished += 1
        if started < n:
            heapq.heappush(q, (t + service, started))
            started += 1
        assert started - finished == len(q)
    return t, finished / t

assert closed(8, 2, 2.0) == (8.0, 1.0)
print(closed(8, 2, 2.0), closed(8, 4, 2.0))
```

继续画“到达/派发、选中、拒绝、完成”四列。如果拒绝一条，K 是否补位？拒绝是否计时间？先写定义，再改模型。真正项目的容量拒绝分支还缺记录，不能把这段练习当其修复已经提交。

练习二 同一组数的不同统计口径

```
import math
xs = list(range(1, 21))
sim_p95 = xs[min(int(len(xs) * .95), len(xs) - 1)]
nearest_rank_p95 = xs[math.ceil(len(xs) * .95) - 1]
equal_cv = (.1 + .9 + .1) / 3
time_cv = (.1 * 1 + .9 * 8 + .1 * 1) / 10
assert (sim_p95, nearest_rank_p95) == (20, 19)
assert abs(time_cv - .74) < 1e-12
print(sim_p95, nearest_rank_p95, equal_cv, time_cv)
```

给 CV 三个快照分别标 t=0、1、9 秒，观察到 10 秒结束。回答等权样本均值为什么约 0.3667，时间加权为什么 0.74。再说明哪些请求属于 TTFT 分母、哪些被拒绝、哪些处在预热期；这比背“看 P95 更准确”更有用。

9 源码阅读顺序

1. TraceRecord 与轨迹适配：逐字段说明来源、单位、可空性和真实路由键。
2. bridge 与 bootstrap：跟到实际 handler 和探针，确认没有旁路。
3. engine：分别手画回放与容量，标出状态写入、标签注入、入堆和拒绝分支。
4. Backend 接口与经验后端：用不同颜色标观测复用、经验采样和参数化回落。
5. redis_shim 与会话记忆：找时钟、TTL 与 Lua 语义边界。
6. metrics、calibrate、CLI：核对每个分母、分位与落盘字段。最后对照独立技术报告，不要只读总结表。

第五章：编码 Agent 平台——把协作、执行与交付讲透

私人面试备战材料，2026-11-02。目标是理解并能推演机制，不是逐句背诵。源实现以已提交 HEAD `6f8f75b02b19757894e26e93a5e3be78178d67b2` 为准；未提交工作不作为完成证据。本文不包含凭证、内部网址、员工信息或业务规模。

标签说明：**【已实现】**表示该版本的实际代码与已运行检查，包括已识别的实现缺口；**【通用知识】**表示语言、系统或实验原理；**【改进建议】**表示尚未交付或尚未验证的方案。讲面试时必须区分这三者。

1. 先用一分钟定位项目与自己

【已实现】编码 Agent 平台面向个人开发者和小团队，把任务计划、实时聊天、工具调用、文件和结果放在同一工作面。用户提交代码任务后，平台管理会话、派发与运行队列，Agent 在工作区调用工具并修改文件；平台展示过程、保存记录并支持结果检查。多会话、网页、编辑器、身份管理和平台底座是团队整体能力，不全部归于个人。

我的主要模块有两块。第一块把上游 **Delm 双 worker 协作语义**适配到平台运行时，用共享看板、身份绑定的 MCP 工具和会话容器协调两名 worker。第二块接入上游 **Harness² 配置调优框架**，用 Python 调优器经 Node runner 实际运行配置变体，再收回执行轨迹和候选产物。长任务可靠性工作包括输入准备、会话隔离、重跑基线、已有文件修改检测、停滞处理和结果回收。

【通用知识】面试叙事顺序应是“谁遇到什么问题→平台怎样工作→我负责哪两块→为何这样设计→如何验证→还缺什么”。不要一开场就讲 claim、MCP、INV/DOM；这些词只有在对方知道系统目标后才有意义。

可以这样开场：“这个平台让一个人或小团队同时使用多个编码 Agent。我负责双 worker 协作适配和 Harness² 执行接入，重点是把任务归属、实际配置、执行记录和可检查产物串起来。除了打通调用链，我还处理了输入漏拷、运行覆盖、修改原文件却被漏判交付，以及同伴停滞被活跃 worker 掩盖的问题。”

【已实现】归因上，Delm 协议与 Harness² 搜索方法均有上游；Harness² 上游代码保留 Apache-2.0 许可。本人确认承担两块工作，Git 直接记录本人集成与修正；基础 Delm 进入团队仓库的提交由协作者署名，后续也有团队修改。可以说“负责适配和工程实现”，不能说“独创整个协议”“全部平台由我开发”。

2. 先分清四层对象

【已实现】整体路径如下，第二条是调优入口，两条最终进入同一执行运行时：

```
Web 任务/聊天 -> Node 服务与运行队列 -> 单 Agent 或 TeamSession -> 工具 -> 工作区
Python 调优器 -> substrate -> Node runner -> 配置物化 -> 同一执行运行时
                                                          |
                                                  事件、轨迹、产物、元数据
```

会话是用户操作和观察的容器；**worker** 是独立模型上下文与工具执行实例；**任务条目**是看板中可认领的工作；**rollout** 是任务、配置和预算给定后的一次执行。一个双 worker rollout 内部有两个 worker，它不等于两个 sweep。

【通用知识】harness 是执行配置组合，包括系统提示、规则、技能、工具指南、脚本、子代理和协作约定。它不是模型权重，也不是训练模型。sweep 组织基线、提案、执行和候选选择。要提高配置搜索质量，既需要好的搜索方法，也需要可信的执行基础设施；本人重点在后者。

【已实现】平台底座使用 Node.js、Express、WebSocket 和文件状态；网页是原生 JavaScript SPA，编辑器使用 CodeMirror。个人模块主要使用 JavaScript/Node、EventEmitter、MCP、Python、CLI/JSON、内容哈希和 JSONL。不要把数据平台的 React/TypeScript 经验误填为该平台前端栈。

3. Node 事件循环：为什么两个 worker 可以交错，但同步认领仍然先到先得

【通用知识】在这个单 Node 进程中，JavaScript 回调通常在同一事件循环上运行。网络、计时和工具执行可以异步等待，等待完成后的回调依次执行。“异步并发”不意味着两个 JavaScript 函数在同一时刻各执行一半。CPU 密集的同步逻辑仍会阻塞其他请求，两个 Agent 的模型等待与工具等待则可以重叠。

`await` 表示暂停当前 async 函数，允许事件循环处理其他工作；它不等于创建线程。Promise 已解决时，后续代码仍通过微任务恢复。`EventEmitter.emit()` 默认同步调用监听器，因此监听器也可能产生重入：尚未退出一次状态更新，就调用到其他逻辑。面试时既不能说“单线程所以没有任何竞态”，也不能把一切交错都归于多线程。

【已实现】`claim` 的“读当前队列→检查 open→检查自己的持有项→追加 claim”之间没有 `await`。同一进程内，第二个普通异步请求进入该函数时会看到第一个已更新的状态，所以先到先得。该结论限定在本进程、本对象和现有同步边界；如果改成多实例或在检查与写入之间加入数据库等待，就必须重新建立原子性。

【已实现：重要源码缺口】MCP 的 `read` handler 当前将 `board.read(...)` 直接交给文本编码器，而 `read` 是 async，返回 Promise。编码器对非字符串调用 `JSON.stringify`，Promise 会序列化为 `{}`。核心状态可能已经推进读取游标，调用者却没有得到视图。`async handler` 本身不会自动等待“作为另一个函数参数传入”的 Promise，外层同步 `try/catch` 也不能据此接住日后的异步拒绝。

静态调用链与真实 board/同形编码器的无模型边界实验确认：没有 `await` 时 `content.text` 为 `"{}"`；先等待结果再编码时能读取真实队列。实验没有调用 SDK 注册的实际 handler，该层回归尚未运行。**不能把 core 的 13 项通过说成真实 MCP 读取链已经验证。**【改进建议】在工具边界显式等待，增加真实 MCP handler 的结果与错误传播回归；本轮未修改源代码。

4. TeamSession：从创建到完成的完整生命周期

【已实现】创建 TeamSession 时生成两个 worker、一个共享 DelmBoard、每 worker 的 MCP server、提示词和事件日志位置。worker 有独立模型上下文，共享文件系统。worker-A 的启动约定偏向建立初始拆分，worker-B 偏向读取看板并接手互补任务。这是提示层的角色安排，任务归属则由运行时约束。

第一条普通用户输入会广播给双方，确保两个 worker 真正开始 SDK 迭代；后续追问转给 lead worker。运行队列通过 `dispatchTask` 也广播任务。如果只构造对象而不派发，worker 不会因为“已创建”就自动工作。新任务还会复位完成标志并开始新的团队周期。

每个 worker 可发布开放条目、认领任务、写发现、附文件、完成自己的条目。聚合事件附带 worker 标识与团队递增序号，便于界面区分来源。出现新开放项时，运行时在下一事件 tick 复查队列，避免把“刚创建、随即被作者自己认领”的任务误作待接手工作；仅唤醒空闲、非作者且无 claim 的 worker。

【通用知识】这里有两种唤醒：等待中的 board.read 可以被 EventEmitter 恢复；完全空闲、没有 SDK 迭代的 worker 没有等待 Promise，仅发看板事件不会让模型自动开始。因此需要显式向空闲 worker 写入继续指令。要解释“事件存在”和“消费者正在等这个事件”的区别。

【已实现】正常完成要求队列曾形成条目、所有条目均 done，并且所有 worker 当前回合均 idle。只看到某个 worker 输出“完成”不够。成功回合结束后，如果 worker 忘记关闭自己的 claim，运行时可用最后一条成功结果摘要自动关闭；失败回合不会这样关闭。

自动 close 解决的是生命周期收尾，不证明测试通过。它依赖成功回合信号，可能把模型自述完成与任务实际完成混在一起。【改进建议】将“执行结束”“条目待验收”“验收通过”分成不同状态，对关键交付要求独立验证。

5. 看板事件、重放与幂等

【已实现】一般事件形状为：

```
{"sequence":17,"timeNs":123456,"actor":"worker-A","kind":"claim","payload":{"item":"item-id"}}
```

sequence 在本对象中递增，作为主要顺序；timeNs 虽然名字像纳秒，实际由 Date.now() 产生，是毫秒，不得用于宣称纳秒时间精度。item/claim/done 派生任务状态，note 表示 FACT/FAIL，body 关联附件，status 表示短状态，session 记录 worker 生命周期，read/wait/apply 等记录读者操作。

任务状态可理解为 Q=Replay(L)。回放 item 建立 open 条目，claim 填 owner 并变 claimed，done 写摘要与 publication。新状态不是通过改写旧日志得到。这样可以追溯“谁何时认领、哪条声明关闭”，但当前队列派生会扫描日志，不等于已有高性能事件数据库。

【已实现】claim 保证“同一条目一个 owner”“每 worker 最多一项未完成认领”。reserve 把创建与认领组合起来；主题匹配忽略大小写及多余空白。主题归一化不是语义去重：不同含义可能同名，同一含义也可能写成不同标题。重复 claim 会返回未赢得认领及当前状态，不应把它一概称作与第一次完全同形的幂等响应。

更严格的发布幂等在 done/note：MCP 根据调用内容与附件生成请求指纹，board 用 (actor, request_id) 查旧 publication。相同标识、相同内容返回同一发布结果；标识相同而内容不同则拒绝。检查旧请求在检查任务已完成之前进行，才能让“响应丢失后的重试”仍成功。

【通用知识】这里的幂等效果是避免重复完成或重复发布，并不意味着每次重试绝无副作用；返回视图时仍可能追加读事件。请求幂等也不自动提供分布式 exactly-once。跨进程需要共享去重存储、事务或唯一约束。

【已实现】日志 JSONL 可以重放任务事件，但写入是 best-effort。更关键的是，附件 _body 持久化只保存路径、字节数、哈希、变化类型和模式，没有完整正文与原始内容。新对象重建后任务仍可为 done，附件正文却不可恢复。独立小实验已确认该差别。【改进建议】将附件正文放内容寻址存储，持久日志引用对象哈希，并对对象落盘与事件追加定义失败恢复顺序。

6. 增量游标与等待：怎样既少发上下文，又不漏事

【已实现】首次读取返回全量；以后以该 actor 最近一条已服务 read 的序号为界，保留变化条目、笔记和状态。read/wait 仍列出未 done 的任务，未变化者压缩为身份、状态、owner 等信息。这不是每次只发新事件，而是“变化的完整对象＋活跃对象的简略提醒”。

游标跟 actor 绑定，A 读取不会消耗 B 的更新。普通成功读取会追加 `read`；等待超时只返回无更新的空壳，并记录 wait，不推进读取游标。否则一个没有取得更新的超时请求会把后续增量起点推走，造成遗漏。注意“已服务”只是服务端生成响应，未必等于客户端可靠收到了响应，真实可靠投递还需要确认语义。

【已实现】等待循环先查历史是否已有唤醒理由，再订阅 wake，醒来后重新检查条件；只有本 actor 以外的相关发布或 peer 结束才有意义。`status` 不直接唤醒。当前检查与注册监听器之间没有 `await`，普通事件循环回调不能恰好插入其中，因此不能机械宣称这里必然存在经典丢唤醒窗口。

【通用知识】若未来在这两步之间增加异步操作，先检查再订阅就可能丢事件。更稳健的通用模式是订阅后再次检查条件，配合序号和结束标志；任何通知都只表示“可能有变化”，消费者仍检查谓词。超时、事件与 stop 都应幂等收尾，注销监听器并取消计时器。

【已实现】当前事件提前唤醒时会移除 listener，但没有同步清除已创建的 timeout，旧 timer 仍会到期再进入已 settled 的 finish，可能多保留事件循环句柄。另一个边界是指定 target 并非严格只等目标：一般 peer 更新也会先触发返回，所以调用者还要确认目标是否满足。【改进建议】补边界测试，不把等待 API 当严格条件变量或可靠消息队列。

7. 文件 apply：哈希保护究竟保护什么

【已实现】对于一个选中附件，记传入哈希为 H、本地哈希为 C、原始哈希为 O、附件注册顺序为 t。board 还知道其他作者曾发布的同路径版本及顺序。判断依次进行：

条件	决策	解释
C=H	unchanged	已是同一内容，无需写
C 属于 peer 已知版本，且注册晚于 t	kept	本地已是较新的 peer 内容，保留
本地缺失，或 C=O 且原始读取无误，或 C 是可替换的已知 peer 版本	write	返回待写内容
其他情况	conflict	不认识的本地分歧需要人工或代理协调

一个选中批次只要有 conflict，就不返回任何待写文件；这是**判定阶段**的整批拒绝。MCP 随后读取本地哈希、调用判定，对获准文件逐个临时写入并 rename。每文件替换比直接覆盖更安全，但多文件不是一个事务；中途出错仍可能只写了一部分。

【通用知识】SHA-256 在这里用于内容身份比较，不证明代码正确，也不是加锁。核心保护范围是经 `board.apply` 的交换；worker 直接用编辑工具写共享工作区时仍可能覆盖。读取哈希到写文件之间若有外部进程修改，还存在 TOCTOU。MCP 同步路径减少同一事件循环中普通回调的插入机会，不消除外部写入。

【已实现】默认 board 没有完整任务原始快照时，原始内容通常未知；未知本地编辑会保守冲突。这减少误覆盖，却可能增加人工协调。【改进建议】按文件明确所有权，使用独立 worktree 与合并、或在应用时比较并交换；不能只换一个更长哈希就称冲突已解决。

8. 停滞检测：观测进度与判定正确性分开

【已实现】运行时为每个 worker 记录最近活动。超过提醒窗口的静默者会进入提醒判断；闲置且仍持 claim 的 worker 被要求关闭任务或说明阻塞，无 claim 者被要求检查开放工作。忙碌 worker 在该提醒分支中会跳过，因此不能说所有静默者都必定收到一条提示。

再次检查时，仍持 claim 且超过失败窗口的 worker，或全员都已超过失败窗口，导致 team-failed。默认提醒/失败窗口分别为 5/8 分钟，可配置。窗口按最近活动计算，不等于提醒后又完整等待 8 分钟；全员静默的兜底也不区分每个工具究竟是否在做长计算。

【通用知识】一条新的模型或工具事件只是活性证据，不是有效进展：Agent 可能不断重复无用动作。相反，长编译可能很久不输出却正常工作。阈值只能降低无限等待风险，不能完全解决分布式系统的故障判定。【改进建议】结合工具运行状态、任务心跳、进展摘要和可取消预算，分别解释“忙”“静默”“无进展”。

9. Harness²：k、2k、k 和最终选择池

【已实现】并行模式先跑 k 个基线 rollout，基于共同证据提出 2k 个 INV/DOM 变体并运行，再合成 k 个 AGG 配置并运行。INV 偏通用执行行为，DOM 偏任务类型知识，AGG 综合不同方向的证据。常规展开共 4k 次 rollout；这是计划数，异常、复用或早停会改变实际完成数。双 worker 模式下，每个 rollout 内又有两名 worker。

最终池只纳入 agg1..aggk；基线与中间 INV/DOM 不在池内。顺序模式的最终标签另为 c1..ck。所以 k=1 时最终只有一个 AGG，选择器没有在 AGG 与基线间择优，也没有内建“全部不如基线就弃选”。不能说“搜索后必然改善”。

【通用知识】匿名呈现产物减少身份标签带来的偏好，但仍有顺序、长度、文风和模型评审偏差。选择器看文件不等于实际执行了验收。适配器当前没有统一评分 rubric，score 返回空值。应把选中候选与基线放到独立测试上比较，并用留出任务检查泛化。

10. Python—Node 契约与配置生效

【已实现】Python adapter 定义任务与产物，substrate 组织执行，Node runner 负责本平台运行时。CLI 传任务目录、harness 目录、run ID、输出位置、模式、超时等，校验契约版本、路径字符与长度。stdout 返回单行 JSON，stderr 保存进度和诊断；写 stdout 后等待刷出再退出，避免管道里只留下半个大 JSON。

批次 ok 表示 runner 完成该批调用，不表示任务成功；runs[].ok 是单次正常结束。rollout_ok 的宽松标准是有运行记录，且正常结束、存在交付物或有工具调用至少一项成立，表示“可分析”，不是正确率。存在目录更不够，因为记录失败前也会建目录。

配置物化把系统提示和规则合并到运行时读取的文件，把技能、工具指南、子代理与脚本映射到约定位置。两 worker 的 startup 与 board guidance 作为参数进入 TeamSession 提示组装。无法映射的组件要被报告，否则调优器可能以为改了配置，执行端却根本没读到。

【已实现】结果保留 workspace、元数据、runner 报告与事件。单 worker 常用 `events.jsonl`；n2 有每 worker 日志，轨迹先渲染 A 再渲染 B。它保留各自步骤，却不是两者按墙钟排序的完整因果时间线。工具调用、观察、文本与错误有统一标签，成功回合重复的最终文本不再渲染一遍。

【改进建议】需要分析并发依赖时另外保留 worker、单调序号、父事件或任务标识；不要从“轨迹中 A 的段落在前”推断“A 一定先做完”。

11. 输入准备、命名空间与产物识别

【已实现】staging 是复制本次任务输入，不是把所有输入直接塞进模型上下文。任务需要的数据可以很大，提案上下文要更小且可读，两种预算不能共用一个武断上限。当前单文件和总量有可配额度，默认分别为 64 MiB 与 512 MiB；超额/复制失败记录跳过清单。依赖、状态和部分二进制扩展另有过滤，因此“额度内必定完整复制任意项目”仍不成立。

缓存键取任务文本的 SHA-1 前缀，并在 adapter 构造时捕获。这样改变任务文字会改变目录，也避免环境变量稍后变化导致同一 adapter 的目录漂移。但相同文字、不同项目文件仍可能复用旧输入；缓存根跨会话共享时还要考虑同文字碰撞、并发构建和文件更新。【改进建议】加入项目版本、输入清单与内容指纹，临时目录完成后原子发布。

运行输出按用户/会话和 namespace 隔离。名称还会进入 run ID，长度是正确性约束；拼接重复前缀曾让标识在模型启动前被拒绝。非标准工作区形状有较粗的回退作用域，因此不能声称任意自定义路径都绝无碰撞。

同一 run ID 重跑时先验证目录严格位于输出根内，再重建工作区，防止旧产物被当成新交付。续跑必须显式把上一轮结果纳入共同 seed，让所有候选从同一输入出发，不能偷偷继承某一路残留。

【已实现】产物判断先看显式交付路径，否则看工作证据和相对 seed 的新增/修改。修改原文件就是很多修 bug 任务的主要产出，不能只找新文件。runner 在物化时记录内容哈希；选择器提取可读文件时也比较内容，排除配置、记账和未改输入。哈希改变只说明发生变化，空改、坏改仍可能入候选，正确性必须独立验收。

12. diff、adopt、rollback、pull 各解决什么

【已实现】diff 比较映射到目标位置并合成后的实际配置；adopt 先保存将覆盖或新建的目标文件状态，再写选中配置；rollback 恢复旧内容，对原先不存在的文件执行删除。它不是整个项目或 Agent 代码的时间旅行，不会撤销外部 API 副作用，也不是跨文件事务。

pull 处理另一件事：从选中 rollout 的 workspace 提取新增或改过的任务文件，排除原始未改输入、配置和记账；默认过滤大文件及二进制，返回跳过清单，也可按候选分目录回收后比较。采纳一个好 harness 和引入一份候选代码应分别审查。只交付配置而丢掉实际代码，是“调优完成”却用户拿不到结果的典型工程断点。

13. 分级追问：先回答主问题，再承受两层深入

以下 14 组用于自测。基础组应在 30 秒内说清；实现组可展开 2 分钟；挑战组先承认边界，再给验证方案。不要把建议伪装为已做。

Q1｜基础：这个平台与你用几个终端开 Agent 有什么不同？

【已实现】它把会话、计划、执行事件、文件和结果放在共享工作面，队列支持组织任务，历史便于重新连接和团队接手。我做的协作与调优模块在这个底座内运行，不是整个底座的作者。

追问一：具体是谁用？个人开发者或小团队同时处理项目任务；不能编造用户量和生产规模。**追问二：你的价值在哪？**把两个 Agent 的任务归属和调优器的实际执行链打通，解决输入、产物与长运行失效，暂不声称量化提效。

Q2｜基础：为什么要双 worker，速度能翻倍吗？

【通用知识】可分工作并行后，时间大致受较慢分支、协调、合并和验证成本支配。两个模型上下文增加探索与交叉检查机会，也增加费用和冲突。强耦合任务不一定更快。

追问一：如何拆？【已实现】A 初始化分工，B 读板认领，运行时限制 claim；不是固定“一个只写代码一个只测试”。**追问二：怎样证明收益？**【改进建议】固定任务、模型、输入和预算，比较单/双 worker 的独立验收、墙钟与成本，重复运行并报告退化。

Q3｜实现：单线程为什么还需要认领协议？

【通用知识】单线程只约束同一回调的执行，两个模型仍会在不同回调里决定做同一件事。认领是业务互斥与归属声明，避免重复工作。同步检查加写入提供本进程的先到先得。

追问一：在检查后 await 会怎样？两个请求可能都看见 open，恢复后各写 owner，需要事务或原子比较更新。**追问二：扩到多实例呢？**【改进建议】共享存储、唯一认领约束或分区单写者；不能继续只靠进程内 Map。

Q4｜实现：什么是事件重放，为什么不直接维护一个对象？

【已实现】日志保留状态如何产生，派生视图可重建任务与归属，便于追查和重连。直接对象也可工作，但容易只剩最终结果而丢掉过程。当前实现扫描日志，写入仍是 best-effort。

追问一：是不是日志存在就什么都能恢复？不是，附件正文未持久化，状态与内容的恢复能力不同。**追问二：怎样改？**【改进建议】先落正文对象再写引用事件，定义崩溃点、校验与回收；增加重启后 expand/apply 测试。

Q5｜实现：claim 与 done 的幂等到底一样吗？

【已实现】重复 claim 返回当前 owner 和未赢得状态；done/note 用请求标识及指纹复用 publication，标识相同而内容变化则报错。两者效果不同，不应统称“所有调用完全幂等”。

追问一：为什么先查旧请求再查任务状态？第一次已成功而响应丢失，重试时任务已 done；先验状态会误拒绝合法重试。**追问二：可以保证 exactly-once 吗？**不能，读日志仍有变化，持久化非事务，多实例也没有共享去重原子性。

Q6｜实现：为什么 actor 不让模型自己传？

【已实现】每个 worker 的 MCP server 在闭包中绑定 actor，工具参数只描述动作。这样不会因模型误填其他身份而操作别人的认领。JSON schema 校验类型，业务层校验 owner，各层职责不同。

追问一：这就是安全隔离吗？不是，共享文件系统和自动工具权限仍有更大边界。**追问二：实际链路最容易漏什么？**async 结果被当普通值编码；本版 read handler 的 Promise→{} 风险说明必须测工具适配层，不能只测 board 方法。

Q7｜实现：增量读取为什么仍带未完成任务？

【已实现】只发变化可能让模型在长上下文中忘掉自己或同伴仍在做什么，故未变活跃项保留紧凑提醒。游标按 actor 独立，超时不推进。

追问一：服务端生成视图等于用户收到了吗？不等于，当前没有完整消费确认；Promise 编码缺陷更会造成已读而未见。**追问二：如何改善？**【改进建议】响应带序号、客户端确认消费位置，允许安全重放；对“重复可接受、漏读不可接受”制定协议。

Q8｜挑战：EventEmitter 等待会不会丢唤醒或泄漏？

【已实现】当前检查与注册间无 await，普通回调不能插入；醒来复查条件，超时/事件通过 settled 防止重复完成。提前唤醒未取消旧 timer，仍有句柄寿命问题。

追问一：以后在检查间加一次 I/O 呢？可能丢唤醒，需订阅后再检查序号。**追问二：怎样测？**【改进建议】覆盖发布在订阅前、订阅后、超时同时、peer ended、stop、连续无关事件，并检查 listener 数与 timer 清理，不只测“等到一条消息”。

Q9｜挑战：有 SHA-256 为什么还会冲突？

【通用知识】哈希只告诉内容是否相同，不给写入排他权。未知本地修改需要拒绝，已知更新 peer 版本应保留。两个直接编辑工具或外部进程仍可越过 apply。

追问一：一个批次拒绝就原子了吗？只在决策层整批拒绝；真正文件写入逐个进行，中途失败可部分生效。**追问二：如何增强？**【改进建议】文件归属、独立工作树、应用前再校验、可恢复写入日志；根据成本选择，不声称现有代码已实现。

Q10｜实现：为什么停滞按 worker 计时？

【已实现】团队只有一个活动时间时，A 不断产生事件会掩盖 B 的停滞。逐 worker 记录能指出谁静默、谁仍持 claim。提醒与失败采用不同阈值，给收尾留下机会。

追问一：成功 idle 自动 close 会不会误报？会有模型成功结束而任务不正确的风险，因此只是生命周期修复。**追问二：长编译被误杀怎么办？**【改进建议】结合工具状态、输出心跳、可配置预算和取消信号；事件频率不能单独代表有效进展。

Q11｜基础到实现：k=2 的搜索到底跑什么，谁被选？

【已实现】常规并行展开是 2 个 baseline、4 个 INV/DOM、2 个 AGG，共 8 次 rollout。最终池只含两个 AGG。双 worker 模式每次内部再有两名 worker，不能把 rollout 数与 worker 数混为一谈。

追问一：基线更好呢？当前选择池不含它，选中结果仍可能更差。**追问二：盲评就客观吗？**【通用知识】不是，应独立运行验收、记录弃选与保留任务结果；盲评只减少一种身份偏差。

Q12｜实现：stdout、stderr 为什么一定分开？

【已实现】Python 要机器解析结果，日志混入 stdout 会破坏 JSON。进度走 stderr，结果写完刷新后退出，避免管道缓冲截断。异常 stdout 也留原始记录，以便诊断。

追问一：进程退出 0 是否成功？只说明调用层状态，还要看每个 rollout、可读产物和任务验收。**追问二：`rollout_ok` 为真呢？**只是存在可分析工作，有工具调用也可为真；绝不能把该字段统计为任务正确率。

Q13｜挑战：相同任务哈希不是已经保证可重复了吗？

【已实现】它只区分任务文本，不标识全部项目内容。输入文件改变而文字不变可能命中旧缓存；跨会话同文字共享还可能产生污染。命名空间解决运行输出范围，不能自动解决任务缓存身份。

追问一：怎么防旧产物伪交付？重跑重建工作区，续跑显式引入公共 seed，内容比较识别真正变化。**追问二：怎样完全复现？**【改进建议】冻结模型版本、输入内容清单、配置、预算与环境，重复采样；外部服务和随机性也要记录。

Q14｜面试收口：测试没全绿，你还怎么证明工作？

【已实现】可以证明已提交实现和已通过的机制测试，但不能证明全部集成稳定。board 13/13、Node 三套 58/58、新机制 Python 子集 35/35；完整 Python 84 项中 58 通过、9 失败、12 错误、5 跳过。35 是子集，不另加。TeamSession 编排测试因缺 SDK 加载终止，并非真实模型实验失败。

追问一：失败都只是环境吗？不是，旧任务接口和 seed fixture 与实现漂移属于源码测试不一致，需修复。**追问二：下一步先做什么？**【改进建议】先补异步 MCP 与附件重启回归，再恢复会话测试、清理旧接口，最后做真实多任务、同预算端到端对照。

14. 两个无需调用模型的练习

练习一：纸笔推演并找出异步边界

【已实现＋通用知识】假设 A 发布 T，B 首读，A claim T，B 增量读取，A done T，B 等待读取。逐步写出 `sequence`、T 的状态、owner 和 B 的 lastRead。说明未变化活跃项为什么压缩，等待超时为何不追加 read；再把 `encode(await read())` 改成 `encode(read())`，预测模型收到什么。

参考答案：以下序号由真实 board 核心探针得到，并非按六个动作随手编号。前提是新建空看板、actors 仅为 A/B、没有附件和额外的 sessionState 调用。第六步调用 `read('B', {waitSeconds: 1})`；前五步严格按题目顺序执行。表中“日志末尾”为动作返回后的 `lastSequence`，“视图序号”为返回视图的 `sequence`；A/B 游标分别是 `lastRead('A')`、`lastRead('B')`，空值表示尚未服务过该读者。

动作	日志末尾	视图序号	A 游标	B 游标	T 状态	owner
A 发布 T	2	1	2	空	open	无
B 首读	3	2	2	3	open	无
A claim T	6	5	6	3	claimed	A
B 增量读	7	6	6	7	claimed	A
A done T	10	9	10	7	done	A
B 等待式读	12	11	10	12	done	A

为什么一次动作可能使序号增加不止一？A 发布时先追加第 1 条 `item`，再由 `served` 生成视图并追加第 2 条 `read(via=publish)`。因此发布动作也推进 A 的读取游标。A 认领会依次产生 `claim`、`status`、`read`，即第 4、5、6 条；A 完成则产生第 8 条 done、第 9 条 `status` 和第 10 条 `read`。读事件本身也是日志事件，但不会改变任务的 open/claimed/done 状态。

`served` 先获取视图，后追加读事件，所以返回的视图序号通常比动作结束后的日志末尾小 1；游标记录的是后追加的读事件序号。例如 B 首读返回视图序号 2，B 的 lastRead 已是 3。这不是漏了一条任务事件，而是读取记账的顺序。游标不能简单取成返回视图序号，更不能取成“第几次调用”。

第六步并不真正睡满一秒：B 的游标为 7，A 的第 8 条 done 尚未被 B 消费，等待函数立即查出 `update`，先记录第 11 条 wait，再由 served 记录第 12 条 read，返回视图序号 11。随后在没有新事件的情况下，再调用一次一秒等待，探针得到日志末尾 13、空壳视图序号 13、B 游标仍为 12；增加的是 wait，不会再增加 read。超时未取得更新，便不推进游标。

未变化活跃项为什么仍要压缩列出？它让读者记得仍在进行的任务与 owner，同时避免反复传 detail 等完整内容；本题第 4 步的 T 刚有 claim 变化，仍按变化对象返回。异步边界部分的答案是 Promise 被同形编码器 JSON 编成 `{}`。更深一问：core read 已推进游标，但模型没收到内容，会不会比“暂时多读一次”更坏？会，后续增量可能不再完整提供已完成信息，所以错误不只是显示为空。

在本章目录运行：

```
node ..\build\agent-sequence-probe.cjs
node .\agent-boundary-probes.cjs
```

脚本只加载已提交快照，不调用模型或 SDK。序号探针将六步状态、超时结果与完整事件序列保存为 `../build/agent-sequence-probe.json`。另一个边界探针写私有临时夹具，日志 `agent-boundary-probes.log` 已保留实跑结果；它的第二段还展示重建后 queue=done、附件 content 不存在，要求你解释“事件元数据恢复”与“交付正文恢复”的区别。这些是核心类与同形编码器的边界检查，不是真实 SDK/MCP 集成回归。

练习二：给失败 rollout 判卷，再推演 apply

【通用知识】手写五行：①无 runs、只有目录；②超时、20 次工具调用、没有变更；③超时但修改了原文件；④正常结束、只写内部记账；⑤正常结束、修改文件且独立测试通过。对每行分别标“可分

析”“有候选”“已验收”。答案：①均否；②可分析但后两项否；③可分析且可能有候选，验收未知；④可分析但没有任务候选；⑤三项都成立。

然后给同一路径设 incoming 哈希 H：本地为 H、缺失、等于 O、较新 peer、未知 X，依次得到 unchanged/write/write/kept/conflict。再加第二个附件 conflict：全批待写列表为空。最后设第一文件已 rename、第二写入失败，解释为什么判定整批通过仍不等于文件系统事务。

【改进建议】独立建小脚本对这些输入断言，不安装 SDK、不发送模型请求。先写预期再读实现；如果推演与代码冲突，保留差异，不为了“练习全绿”篡改事实。

15. 本地阅读路线、实测证据与最薄弱处

【已实现】阅读入口是本章，而不是一次打开所有文件。干净源快照位于 D:/博士后出站考核/输出/agent 报告证据_20261102/head-snapshot/；原始源仓库保留不动。建议按以下问题走：

1. **平台与职责**：先读本仓库专题 docs/delm-harness2-technical-report.md，区分系统整体与个人边界。
2. **一个任务怎样完成**：server/src/team-session.js 的构造、write/dispatchTask、_kickIdleWorkersOnOpenItem、_maybeComplete、_stallCheck。
3. **状态为什么可信**：board.js 的 append/queueState/claim/reserve/done/replayed/served/read/apply；再看 board-mcp.js 如何编码 async 返回值。
4. **配置怎样真正执行**：bench/harness2/CONTRACT.md、lib/engine.js、runner 的 realRun，以及 lib/transcript.js。
5. **调优怎样组织**：vendor/harness2/harness2/core/recursion.py 与 selection.py；逐项数 k/2k/k 和 selection_pool。
6. **输入与结果**：adapter 的 _compute_stage_key/_stage/deliverable_files/rollout_ok，再看 harness-result.js 的 diff/adopt/rollback/pull。

提交阅读锚点：533bd9a 为集成；1b51558 为任务文本键；bf3a2a6 为会话隔离；c708225/a51c913 为输入额度及存储根；98dbeed/8e1ead7 为可分析运行；7e7520d/d1939c2 为逐 worker 停滞、重跑基线和 seed 内容。每次读提交都回答“原错误→修正判据→新增测试→仍存边界”，不要背提交数量。

真实测试日志位于 D:/博士后出站考核/输出/agent 报告证据_20261102/：board.log、harness-node.log、harness-python-targeted.log、harness-python-all.log、team-session.log，运行脚本为 run_checks.py。当面试官追问可出示离线检查范围，而不是回避未通过部分。缺 SDK 阻断编排测试和源码测试漂移是两个不同问题，不能混说“环境原因”。

三处要练熟：①异步：Board 单测不覆盖 MCP 包装，能解释缺 await 并设计边界回归，当前缺口未修。②一致性：事件重放不恢复正文，apply 非事务，按崩溃、并发写与重试推演。③效果：完整回归未全过；真实模型同预算对照和泛化待做；rollout_ok、被选中或提交量不能证明质量。

06 跨项目追问与闭卷模拟

本章把各项目共同涉及的身份、状态、并发、异步和验证问题连起来。跨项目比较用于展示判断力，不表示这些平台实际部署成一套系统。下面所有扩展方案均需按项目条件验证。

1 最容易混淆的六组概念

1.1 任务完成 模型调用完成 与交付正确

【通用知识】进程退出说明进程生命周期，HTTP 成功说明某层协议状态，工具调用说明发生了动作，产物存在说明有文件，验收通过才说明指定要求被验证。四层应分别留证据。Agent 的 rollout_ok 故意保留可分析的失败过程；不能把它的通过率改名为任务成功率。类似地，元数据发送成功不表示搜索索引已经可见，权重接收完成也不表示推理 selector 已经切换。

1.2 进程内锁 与分布式协调

【通用知识】同进程按 URN 加锁可以减少注册重入，同一 Node 对象的同步 claim 检查可以维护本地归属，但另一个服务副本没有参与这把锁。要扩展为多实例，需要共享唯一约束、版本比较、事务或单写者分区等机制。选择哪一种取决于冲突范围、失败恢复和延迟要求，不能只说“换 Redis 锁”就结束。

【改进建议】若采用租约，继续回答租约过期但旧持有者仍执行怎么办，是否需要 fencing token，以及如何把令牌带到真正写入处。锁只保护检查过程而最终写入不校验版本，仍会有过期持有者覆盖。当前项目没有统一采用这一整套方案，不要当成已交付能力。

1.3 批量 与原子性

【通用知识】Redis pipeline 用于合并请求/响应、降低往返成本；它本身不等于一段条件读改写的原子事务。MULTI/EXEC 提供一组命令不被其他客户端穿插的执行语义，执行期错误也不能按关系数据库回滚直觉理解；Lua 脚本可把条件与更新放在服务器侧执行，仍需考虑运行时间、键所在分片及故障边界。参阅 Redis pipeline、Redis transactions 和 Redis Lua。

相同区别也出现在推理与 Agent：把传输指令编成一批，不证明 HCCL 底层一次原子发送；一次 apply 判定全部通过，不证明多个文件已经事务性替换；先做 snapshot，不代表后续任意失败均能完整自动恢复。

1.4 异步函数 与异步结果

【通用知识】调用 Python 协程不等于已经执行完成；要 await、创建 task 或由运行器驱动。Node async 函数返回 Promise，把它当普通对象传给 JSON 编码器不会自动得到完成后的视图。一个外层 async 函数也不会自动 await 其所有参数里的 Promise。相关原理见 Python 3.12 协程与任务。

Node EventEmitter 默认同步执行监听器，因而应注意回调重入、订阅清理和错误传播，而不能笼统说“事件是异步的所以安全”。见 Node 事件机制。这些是语言语义；是否影响项目，要继续读当前包装和调用链。本手册 Agent 章用真实 board 与同形编码器说明遗漏 await 的风险，真实 SDK handler 回归尚未执行。

1.5 身份事实 访问凭证 与业务权限

【通用知识】actor 标明谁发起操作，token 提供访问凭证，角色/领域策略决定允许做什么。服务级 token 有能力调用后端，不表示每个最终用户都拥有同样权限。前端隐藏按钮不是服务端授权。MCP server 绑定 actor 可以防止模型随意伪造看板身份，但不能自动限制共享文件系统上的全部编辑动作。

数据平台应追踪登录如何穿过 GMS、Play 与扩展 API，注册工具如何取到 actor 并检查 can_register；读取工具则要核对其实际使用的凭证和可见范围。Agent 的本地 actor 绑定和企业 SSO 不是一个机制，不能用相同术语模糊过去。

1.6 版本 哈希 与有效性

【通用知识】哈希帮助比较字节身份，不证明逻辑正确。任务文本哈希不能自动覆盖整个输入工程；文件哈希不能防止“检查后又被其他进程改”；代码 commit 不能代替完整依赖和配置。说“可复现”时至少要有输入、版本、解析后配置、随机源、依赖和结果口径，并说明哪些已归档、哪些仍缺。

2 八组跨项目追问

X01 这些项目中哪些是你独立做的

答：我用模块和决策回答。OmniInfer 是团队框架，我重点参与专家均衡的算法工程、运行时、工具和适配；独立 Omni-EPLB 是相关能力演进，我的后续贡献有通信 context 等。数据平台我做扩展 API/注册 SDK、身份权限、GMS/Play/React 和 Data Agent。网关路由与同事共同设计，状态刷新和重复计数修复有直接提交。Delm/Harness² 来自上游，我承担运行时适配和可靠执行。

追问一：如何证明？准备一个个人补丁、一个输入到输出的调用链、一个验证记录，说明具体做了哪项选择。追问二：团队收益如何归你？团队/组件结果支持总体价值，个人贡献另以模块证据界定，不把系统全部收益分给某一个补丁。

X02 你更偏研究还是工程

答：先讲岗位相关主线，再给同一工作的两面。专家均衡中我能说明目标、约束与贪心性质边界，也能说明 Python/C++、设备权重、统计和安全点；网关与 Agent 更多是状态、接口和可靠交付。它们有共同的问题建模方法，但不是把所有项目都包装成算法创新。

追问一：数学背景具体帮了什么？准确界定目标、区分最优子问题与组合启发式、构造反例和验证条件。追问二：工程判断体现在哪？发现理论假设进入真实系统后会被缓存、观测误差、设备同步或异常生命周期打破，并给出可测的接口与不变量。

X03 如果规模扩大十倍 你先改什么

答：先问是哪一项负载扩大，测哪个瓶颈。推理可能是通信或内存；网关可能是状态访问、后端拥塞或监控刷新；元数据可能是索引延迟与写入路径；Agent 可能是模型预算、工作区隔离和长期运行。先分解测量，再选择批量、分区、缓存或隔离。

追问一：为什么不直接加副本？副本不解决共享瓶颈，并可能放大竞争与状态协调成本。追问二：怎样证明优化有效？固定输入和资源口径，比较吞吐、尾时延、错误/拒绝、资源成本；确认没有以丢记录或删除语义换来表面加速。

X04 什么是你最重要的正确性不变量

答：按项目给具体对象。专家更新要求执行映射不指向未就绪权重；注册要求身份与 URN/依赖规则一致；网关计数要求预留和释放口径一致、每条监控事件不重复处理；看板要求只有 owner 能完成 claim；调优要求候选起点一致、产物来自本次实际运行。

追问一：如何检查？对最小反例记录状态变化，注入取消/重试/版本切换并对账。追问二：已经全保证了吗？明确指出当前缺口，例如内存 Redis 语义、MCP 异步包装和附件正文持久性，不能从设计目标推导实现已经正确。

X05 一次故障怎么完整讲

答：按照触发、现象、定位、根因、修改、回归、剩余边界讲。可选择本人直接修复的重复计数：特定日志过滤分支手动调用 Prometheus，而正常成功处理还会再调独立 callback；修复职责分工，验证请求和 token 计量路径。不要只说“删了一行代码”。

追问一：为何先怀疑指标链？对比真实请求、日志与 counter 的关系，寻找固定倍率及特定调用分支。追问二：现在所有路径都对了吗？本手册未重跑全部线上矩阵，应区分原修复证据与现在建议补充的流式/失败/重试回归。

X06 为什么不用一个更简单的方案

答：承认简单方案是重要基线，再说明它在哪个约束失效。轮询简单但不保留前缀机会；绝对最小负载可能受旧快照影响；纯自然语言多 Agent 分工缺少可执行任务归属；所有元数据写入统一强同步会拖住高频过程上报。当前方案也有复杂性成本，必须用问题而非术语数量证明价值。

追问一：如果简单方案表现更好？按证据选择，不因为自己实现复杂方案就继续维护。追问二：怎么做对照？相同输入、资源、预算与失败定义，保留退化场景，避免给自己的方案使用更多标签或不同缓存条件。

X07 测试通过为什么线上仍可能失败

答：测试覆盖的是给定边界。mock 不模拟设备异步，core board 不覆盖 MCP 编码，dry-run 不覆盖真实模型，内存替身不覆盖 TTL/Lua，exists 轮询不检查所有字段和索引。应画依赖边界，在每一条跨边界调用补契约和失败回归。

追问一：测试怎么排优先级？先测试会改变结论或破坏状态的边界，如丢 await、重复回调、重启恢复和取消释放。追问二：是否需要穷举？不现实，先用不变量、小规模精确枚举、性质测试和代表性故障，再做集成与真实负载；不同方法各有覆盖范围。

X08 问到暂时记不清的实现细节怎么办

答：先说自己确定的对象和主线，再区分需要查版本的部分。例如“当前运行时维护历史统计，但这个版本候选主路径读的是本轮 delta；具体窗口参数我会以函数调用链核对，不能只凭变量名保证十窗口预测”。比编一个默认值可靠。

追问一：这会显得不熟吗？真正危险的是说出互相矛盾的保证；应把常用主线、本人决策和关键边界学到能闭卷解释。追问二：当场发现缺陷怎么办？说明触发条件与影响范围，给最小验证和修复方案，明确尚未修复，不用“这是 feature”回避。

3 十二个快速自检

问题	应立即说出的要点
总体框架和独立 SDK 的关系	同源能力独立交付，不替代整个框架，不重复统计原创
多个副本是否改变逻辑专家	保留模型语义，改变物理执行位置和工作分担
atomic ready 能否代替设备同步	不能，主机发布、设备完成和跨 rank 握手各管一层
GraphQL 是否天然强一致	不是，要看 resolver 和实际读取的数据源
DataHub MCP 与 Agent MCP 是否相同	一个是元数据变更提案，一个是工具上下文协议
日志过滤是否应该手动补指标	看完整回调链，避免与已有独立 callback 重复
健康会话命中必读负载吗	实际路由可直接返回；重选路径再读取
路由 TTL 是否等于 KV 驻留	不是，映射寿命与设备缓存状态不同
高回放一致率能否当预测能力	标签注入与观测复用时不能独立证明
单线程是否没有竞态	仍有 await 交错、同步回调重入和外部文件写入
文件 sha256 是否就是事务	只是内容比较，检查到替换仍有竞态与多文件失败边界
被 Harness[2] 选中是否优于 baseline	当前最终池不含 baseline，且无内建弃选，需独立对照

4 一小时闭卷模拟

前 5 分钟讲开场与项目地图，听者只能问“这个项目到底做什么”“你负责哪块”。如果回答需要不断解释缩写，回到每章一分钟入口重新练。

随后 15 分钟选一个主项目深挖。推荐推理方向从单 token 执行到布局切换，或者 Agent 方向从任务输入到产物回收。要求画状态和接口，不允许只画产品模块名。

接着 10 分钟讲另一个跨栈项目，重点说明技术选择与信任边界。数据平台可以从登录、注册到可查询；网关可以从健康候选、会话到负载和计量。

再用 10 分钟做故障题：取消发生在计数增加之后、某 rank 停止握手、事件已落盘但正文没落盘、注册已发布但查询超时，任选两项。先解释当前行为，再给改进。

最后 20 分钟分为练习 10 分钟和复盘 10 分钟。对每个答不出的点记录“缺概念、缺调用链、缺代码细节、缺验证、说过满”哪一种。当天只补最薄弱的两项，再复讲同一个问题，避免无目标刷题。

评分采用 0—3：0 是只认得名词，1 是能讲正常路径，2 是能讲输入输出与关键约束，3 是能解释失败边界、替代方案与验证。每个项目至少做到 2，主项目关键链路做到 3。分数是自我练习工具，不是面试通过率预测。

5 面试前最后核对

能闭卷画五条主线；能准确区分个人、协作、团队与上游；能说清至少一个本人直接修复的问题；能推演一个小反例；能指出各项目一项真实限制和下一步验证。最后再看源码版本，防止背熟了旧注释，却回答错当前函数体。

更多本地材料：面试 Beamer 与讲述提示、Simulator 独立报告、Delm 与 Harness² 独立报告。公开原理参考只用于解释通用语义，不代替项目调用链和版本证据。